

Privacy-Preserving Continuous Process Discovery

Profilprojekt Anwendungsforschung (INF-PM-FPA)

Fabian Sandkuhl

fabian.sandkuhl@mailbox.tu-dresden.de

Born on: 28.12.2001 in Berlin

Course: M. Sc. Informatik

Matriculation number: 5116793

Matriculation year: 2022

27th of July 2026

Advisor

M. Sc. Maximilian Weisenseel

Supervisor

Prof. Dr. Florian Tschorsch

Statement of authorship

I hereby certify that I have authored this document entitled *Privacy-Preserving Continuous Process Discovery* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 27th of July 2026

Fabian Sandkuhl

Declaration of the usage of Artificial Intelligence

I hereby declare that the core conceptual design and the majority of the software implementation for this project were developed independently. Generative AI tools (specifically Claude / Gemini) were utilized during the research process to support the analysis of the hyperparameter configurations, the implementation of data visualization scripts, and for checking the correctness and writing style of this report. However, all research, analysis and conclusions presented in this work are the result of my own intellectual effort and remain entirely my own work.

Process Discovery on continuous event streams enables organizations to gain real-time insights about their processes. However, as with static process discovery algorithms, there are privacy concerns, since process mining involves highly sensitive personal data. Differential Privacy provides strong mathematical guarantees to protect individual privacy, but extending it to continuous observations is challenging, especially when aiming to protect all sensitive data of a user.

To address these challenges, this work presents a novel differentially private continuous Directly Follows Graph discovery pipeline. The pipeline enforces user-level differential privacy while maintaining a bounded memory footprint and model utility. To limit global sensitivity, we combine case frequency counting, temporal sliding windows gated by Bloom Filters and trace truncation. Furthermore, a dynamic budget recycling mechanism is integrated to enable worst case infinite publications over time without budget depletion. To recover structural model utility from the injected noise, we apply post processing network flow conservation and start activity filtering.

We empirically evaluate our pipeline across a grid of 54 hyperparameter configurations using four real world event logs. Our results demonstrate that the pipeline successfully generates private process models that converge close to the non private baseline. Furthermore, we show that the parameter choice (e.g., publishing schedule) interact in complex ways, indicating that their joint configuration is critical to balance the trade-off between utility and privacy guarantees.

Contents

1	Introduction	7
2	Background	8
2.1	Process Mining Foundations	8
2.1.1	Event Logs	8
2.1.2	Event Stream	9
2.1.3	Process Discovery	10
2.1.4	Model Quality Dimensions	10
2.2	Privacy	13
2.2.1	Differential Privacy	13
2.2.2	Sensitivity	14
2.2.3	Composition and Post-Processing Theorems	14
2.2.4	Privacy Levels	14
3	Methodology	16
3.1	Graph Representation	17
3.2	Bounding User Influence: Sensitivity and Windowing	17
3.2.1	Per-Transition Bounding: Case Frequency	17
3.2.2	Temporal Bounding: w-Event Privacy and Bloom Filter Gating	17
3.2.3	Global Bounding: Trace Truncation	18
3.2.4	Publishing Schedule	19
3.2.5	Laplace Noise Injection	20
3.3	Utility Improvement via Post-Processing	20
3.3.1	Accumulated DFG and Structural Correction	20
3.4	Summary Methodology	20
4	Design and Implementation	22
4.1	System Overview	22
4.1.1	Streaming Pipeline	22
4.2	Experimental Framework	24
4.2.1	Dynamic Parameter Grid Generation	24
4.2.2	Baseline Generation	24
4.2.3	Evaluation Metrics & Process Discovery	24
5	Evaluation and Discussion	26
5.1	Experimental Setup	26
5.2	Evaluation Results - Event Count Based	27

5.3	Evaluation Results - Time Based Mode	30
5.3.1	Budget Consumption and Mean Relative Error (MRE)	31
5.4	Parameter Sensitivity and Model Quality Heatmaps	33
5.5	Discussion	35
5.5.1	Privacy vs. Utility Trade-off	35
5.5.2	Post-Processing and Miner Robustness	36
5.5.3	Memory Efficiency	36
5.5.4	Limitations	36
5.5.5	Future Work	36
5.5.6	Guidelines for Practitioners	37
5.5.7	Summary	37
6	Related Work	38
7	Conclusion	39
8	Appendix	43

1 Introduction

Process Discovery is a core field in the discipline of process mining, providing techniques to automatically construct process models from event data [3, 42]. Traditionally, process discovery methods operate on static event logs which are historical datasets storing recorded process executions [11, 36]. However, in modern operational environments, data is generated continuously at high velocity and volume [42]. Traditional offline discovery algorithms fail to handle such continuous event streams due to memory constraints and unbounded execution times [42, 27], creating a need for streaming process discovery algorithms that analyze data “on the fly” to provide real-time operational insights.

Event logs as well as event streams frequently record highly sensitive personal activities, such as patient pathways in healthcare or transactional data in financial systems [30, 21, 33]. Differential Privacy has established itself as a good measure for privacy preserving data analysis, guaranteeing that the inclusion or exclusion of a single individual’s data cannot be inferred from published outputs [18, 16]. While Differential Privacy has been successfully applied to static event logs (e.g. [30]) and discovery algorithms (e.g. [37]), extending DP guarantees to continuous event streams presents fundamental mathematical challenges [19].

In a continuous streaming environment, data streams are potentially infinite, leading to unbounded global sensitivity and fast privacy budget exhaustion under standard sequential composition [19]. Repeatedly publishing updated process models, such as Directly-Follows Graphs, over time would either exhaust the privacy budget almost immediately or require injecting massive noise [19]. Furthermore, protecting entire user traces (user-level privacy) over an infinite stream requires strict architectural mechanisms to prevent cross-window privacy leakage. To address these challenges, this work presents a differentially private continuous Directly Follows Graph discovery pipeline for event streams that guarantees strict user-level Differential Privacy while mostly preserving structural model utility and memory efficiency. The proposed methodology combines Case Frequency counting inspired by Schulze et al. in [37], w -event sliding windows as in w -event privacy [24], Bloom Filter [8] gating, and trace truncation (L) to bind global sensitivity to $\Delta f = \min(A^2 + 2A, L + 1)$. We integrate a dynamic budget reclamation strategy [24] to support indefinite publishing without budget depletion, and apply post-processing flow correction to restore graph utility.

The remainder of this report is organized as follows: Chapter 2 provides the necessary theoretical background on process mining, Directly Follows Graphs and Differential Privacy. Chapter 3 details our methodology for bounding sensitivity, continuous budget management, and post-processing. Chapter 4 describes the streaming architecture and implementation, while Chapter 5 presents an empirical evaluation across multiple real-world event logs and presents our findings. Chapter 6 presents related work and finally, chapter 7 concludes this report.

2 Background

This chapter aims to provide the needed theoretical background in order to follow the methodology of this report. We first start by introducing Process Mining or more concretely Process Discovery and the necessary models and algorithms of this field. Afterwards, we conclude this chapter with a dive into Differential Privacy, its properties and the various privacy levels.

2.1 Process Mining Foundations

Process Mining is an emerging discipline which bridges the gap between traditional data science and business process management, designed to provide insights into operational processes by analyzing event data [3]. It enables organizations to discover, monitor, and improve real-world operational processes by extracting knowledge from event logs [2, 3].

2.1.1 Event Logs

Event logs serve as the foundation of all static process discovery techniques. They provide the raw data necessary to analyze, understand, and ultimately discover a process model. As event logs are the foundation to process discovery, it's necessary to define the components of event logs in order to understand the influence of each part. The definitions are according to van der Aalst [2, 3]:

Definition 2.1.1 (Event). *An event represents a single activity or action occurring in a process. Each event is a recorded instance of something happening i.e. 'payment received' or 'order shipped'. We denote E as the universe of events.*

Definition 2.1.2 (Attributes). *Attributes are pieces of information associated with an event. For example, most events are associated with attributes like 'timestamp' (when it happened), 'activity name' (what happened) and resource (who or what performed it).*

Definition 2.1.3 (Case). *A case (treated to be the same as 'trace' in this work) represents an individual process instance. It is a sequence of events that together describe the journey of one instance through the process, for instance a patient's treatment in a hospital. In this work, we associate a trace to be directly linkable to a user.*

Furthermore, traces consists of events and additional attributes associated with the case as a whole. Each case has a unique identifier that links associated events and attributes to the trace.

Definition 2.1.4 (Event Log). *An event log is defined as a set of traces $L \subseteq \mathcal{U}$. It is structured data containing all recorded events, including their attributes, and cases they belong to, including their general attributes.*

For example in table 2.1 an example event log is presented containing two traces with in sum 6 events. The traces can be identified through their differing case id. Every event has three event attributes timestamp, activity and resource since the values aren't equal throughout the trace. Additionally, has this event log one case attribute place, because the value of the case attribute stays the same in one trace.

Case ID	Activity	Timestamp	Resource	Place
1	A	2024-02-01 08:30	Alice	classroom
1	B	2024-02-01 09:00	Bob	classroom
1	C	2024-02-01 10:15	Charlie	classroom
2	A	2024-02-02 09:00	Alice	Playground
2	C	2024-02-02 11:45	Charlie	Playground
2	D	2024-02-02 12:30	Dave	Playground

Table 2.1: Example Event Log

Event logs we use in this work are stored in the so called eXtensible event stream format [23]. It is a standardized format that enables the digital storage of event logs via an XML-based notation. This standardization supports various data types and simplifies the identification of attributes through predefined extensions. Because XES data is hierarchically [23] structured into logs, traces (cases) and events, the attributes are assigns to specific levels. The most important attributes for this work are "case:concept:name", which serves as the identifier for the case ID at the trace level, alongside "concept:name" (activity identifier) and "time:timestamp" (event timestamp) at event level. In the next section, we will expand our understanding of event logs and introduce event streams. Simply put, event streams can be viewed as continuous, unbounded (infinitely large) sequences of events, as opposed to static, finite event logs [42, 11].

2.1.2 Event Stream

An event stream S is an infinite sequence of unique events. Formally, it is expressed as a sequence [42]:

$$S \in E^*$$

subject to the domain and uniqueness constraints according to Zelst [42]:

$$\text{dom}(S) = N \wedge \forall i, j \in N : S(i) = S(j) \Rightarrow i = j \quad (2.1)$$

where E denotes the universe of events and for any $i \in N$, $S(i)$ specifically maps to the i th event observed sequentially over the stream.

This work assumes the stream to follow the Insertion-Model or "Cash-Register" Model as described by Muthukrishnan [32] and by Zelst [42], in which an event is immutable after it is received on the stream, i.e. it can not be updated, deleted or changed (as opposed to an insert-delete model). Furthermore, events are assumed to arrive in a strictly sequential, atomic fashion. Unlike static event logs, an event stream forces incremental analysis [11]. Because simple storage of an infinite stream leads to infinite memory consumption, a stream based system must enforce memory bounds [42, 11]. Once a raw event arrives, it has to be discarded immediately or aggregated into an internal, finite data structure, it explicitly can't be reviewed in its raw initial form again [11].

2.1.3 Process Discovery

A way of aggregating an event into a data structure is by using it to discover a model of the observed stream or log, as we do in Process Discovery. Process discovery is a primary task in the field of process mining [2]. It traditionally focuses on extracting process models from real-world event logs without any prior knowledge about the underlying process. By analyzing the sequence of recorded events, process discovery techniques generate a (visual) representation of how a process operates in practice [3]. The goal of the models is to provide valuable insights into the actual behavior of the recorded process, including their structure, variations and inefficiencies [2]. Using event data as foundation for process discovery ensures the resulting model to reflect real-world behavior and can therefore reveal differences between expected and actual expectation [3].

Achieving such an accurate model is challenging. Primarily, the recorded event log contains only example behavior of the operational process, which may include infrequent paths that do not necessarily represent the process as a whole [2]. The challenge therefore is to create an accurate representation of a process's control flow based on the available example behavior. An additional challenge in this work is our focus on streaming process discovery, i.e. instead of a static event log, we use a dynamic event stream as our data foundation. Streaming process discovery introduces specific constraints [11, 42]. Most notably, the chosen process discovery algorithms must be memory-efficient since the input is potentially infinite. They should ideally guarantee an upper bound on memory consumption [42, 11]. According to Burratin [11], there are several strategies to address this challenge. A common approach, which we employ here, is a window based model. This technique uses a window of fixed capacity to store recently executed events (although other strategies can be applied, for further details, please see [42]). Further details regarding our approach are discussed in the Methodology chapter.

We aim to start with a fundamental process model that serves as a foundation for many other process mining tasks and structures, known as Directly Follows Graph [3].

Directly Follows Graphs

A Directly Follows Graph (DFG) is a simple process model, where the nodes represent the different activities of a process and the directed edges represent the directly follow relations, i.e. which activity is followed by another activity [2, 3, 42]. Additionally, edges are typically weighted by their frequency, capturing how often a specific directly-follows relation occurs in the event data. A DFG can be modeled to include a special artificial start-activity node which serves as source to the Graph and a special artificial end-activity node which serves as sink, i.e. all edges from the start node are outgoing edges and all edges connected to the end node are incoming edges [3]. While DFGs are among the simplest of all process models [3, 27], they are highly significant. They serve as the underlying structure for numerous algorithms and can be transformed into more complex process representations, such as process trees or Petri nets, if required [2].

Figure 2.1 is an example of one DFG with 4 activities (a_1, a_2, a_3, a_4) and two dummy activities for start and end activities (a_{START}, a_{END}). Each edge is directed and has a frequency count, which represents how often certain activity follows another. For instance, the activity a_{END} follows 60 times after the activity a_3 . We explicitly define here, that DFG edges can be negative as well as positive.

2.1.4 Model Quality Dimensions

In process discovery, utility is not explicitly defined in the literature. Instead, it can be understood as the usefulness or applicability of a process model, reflecting how well it meets an

Sample Directly-Follows Graph (DFG) with Edge Frequencies

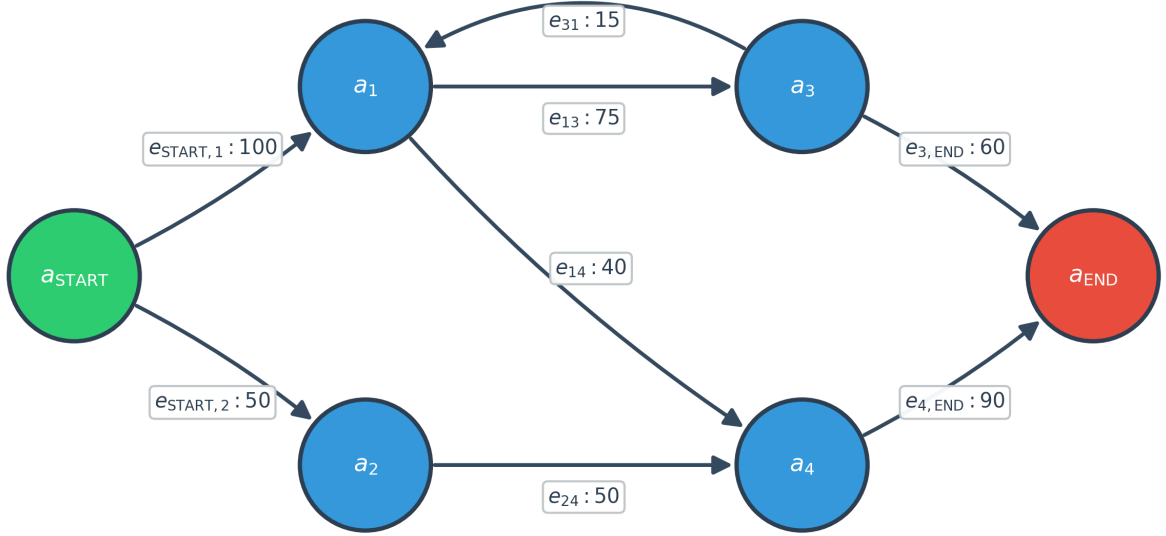


Figure 2.1: Directly Follows Graph with sample edges and Frequencies

organization's needs. This research therefore approaches utility as an implicit aggregation of various quality dimensions. When evaluating the quality of process models, several core metrics are commonly considered. The most important quality dimensions are **precision**, **fitness**, **generality** and **simplicity** [3, 38, 9]. This work considers additional dimensions beside the quality of discovered process models, to read more about this please see the implementation 4 and evaluation chapter 5. In the remainder of this section, all previously mentioned quality metrics and the methods for quantifying them in Petri Nets are explained. The explanation focuses around Petri nets as the tools in the chosen software pm4py [7] offers implementations for evaluating those dimensions.

Fitness

Fitness or Recall is one the most important quality metric and measures how well a process model reproduces the behavior recorded in an event log [9]. There are two major approaches on assessing this metric. The first approach uses an alignment-based technique according to Buijs et al. [9] that matches as many events as in the log as possible with activities in the model. When mismatches occur, penalties are applied for skipping events or inserting activities. The replay fitness score is computed as:

$$Q_f = 1 - \frac{\text{cost for aligning model and event log}}{\text{cost to align event log on model with no synchronous moves}}$$

The denominator represents the worst-case scenario where no alignment is possible, ensuring the fitness score is normalized between 0 and 1. The replay fitness score 1 means a perfect replay of the event log in the model. The alignment computation is computationally intensive, but is a robust method for evaluating how well a model captures observed behavior.

The second approach according to Berti et. al. [6] is assessing replay fitness through token-

based replay. It simulates the movement of token through the Petri net, comparing observed traces in the event log with the model's behavior. For each trace, tokens are produced, consumed and synchronized with transitions, while deviations are recorded as missing or remaining tokens. A score is computed based on the ratio of correctly consumed tokens to the total number of produced and consumed tokens. This method is computationally efficient, especially in comparison to alignment based methods for large logs and complex models, but has some limitations as well. In some scenarios is the performance worse than other replay techniques and there is no proposed fitness or termination guarantee.

Precision

In general, precision indicates to which extent a process model avoids behavior that is not observed in the event log [3]. This can again be quantified by using an alignment based method and a method based on token replay. The alignment-based approach according to Adrian-syah et.al. [4] involves aligning each trace from the log to sequences in the model and identifying situations where the model allows more behavior than the log reflects. From this an automaton is build by using the alignments, capturing both observed and model-allowed behaviors. For each state, the executed (log based) actions are compared against the available (model-allowed) actions. A precision score is then computed as the weighted proportion of executed actions relative to the allowed actions across all states. This method ensures robustness against deviations and incomplete logs, offering a normalized precision value between 0 and 1.

Another method according to Munoz-Garma and Carmona [31] is based on token replay. Similarly to token based replay in the fitness section, the method uses tokens to simulate the movement through the model as traces from the event log are replayed. During replay, tokens are produced and consumed by transitions in the Petri net, reflecting both the observed and allowed behavior. If the model permits actions not observed in the log then those are marked as escaping edges. Quantifying precision is done by comparing the amount of escaping edges to the total allowable behavior in the model. The fewer escaping edges the higher the precision score.

Generality

The following section summarizes the content of Buijs et al. from [9]. Generality should indicate whether a process model describes the actual real-world system and whether the model does 'overfit' the recorded event log. In other words, generality should indicate whether the model can describe unrecorded behavior which can appear in the observed system. This is an important quality dimension as the previous dimensions fitness and precision only consider the relationship between recorded event log and process model and not whether the model likely fits the actual process (including not recorded behavior). It is assessed based on the frequency of transitions and places being used in the process model during the replay of the event log, using alignment metrics from replay fitness. Transitions that are frequently executed visited indicate correct behavior, while transitions or places rarely utilized suggest poor generality, as they may reflect unused or extraneous parts of the model. The generality score is calculated by summing the normalized values of the inverse square root of transition execution counts, then averaging over all transitions in the net. This normalization prioritizes significant improvements at lower execution counts. Despite challenges in precise quantification, the measure aligns with intuitive expectations for generality quality.

Simplicity

At last simplicity refers to the minimalism in the structure of the process model, ensuring it is not overly complex while still accurately describing the behavior in the event log [39]. A simple model avoids redundant tasks, unnecessary connections or pathways that could obscure the true process dynamics. It provides clarity by using the fewest possible elements to process comprehensively. Put simply, a model is simple if it's not too complex nor too big.

In summary, utility in process discovery is an implicit aggregation of various quality dimensions that determine how well a process model meets an organization's needs. Among the core utility metrics, fitness evaluates how accurately a model reproduces the observed event log, while precision ensures the model does not allow excessive behavior beyond what is recorded. Simplicity emphasizes a minimalist structure that maintains clarity without unnecessary complexity, and generality assesses whether the model can generalize beyond the recorded traces without overfitting. Each of these dimensions plays a crucial role in balancing model accuracy, interpretability, and practical applicability.

The following section explores the privacy model used in this work with a main focus on Differential Privacy.

2.2 Privacy

In general, privacy is seen as the right of individuals to control how, when, and to what extent information about them is communicated to others e.g. organizations or governments [41]. This fundamental concept of privacy becomes particularly significant in the context of Process Mining, where privacy concerns arise due to the sensitive nature of the data involved, as event logs often contain personal or confidential information about individuals, transaction or business operations [30, 33].

2.2.1 Differential Privacy

Differential Privacy (DP) provides a mathematical framework for quantifying and minimizing privacy risks when analyzing data. According to Dwork [18], it ensures that the output of a randomized algorithm remains statistically almost identical whether any single individual's data is included in the dataset or not.

To formalize this, we must define the concept of neighboring datasets. The Definition is according to Dwork [17]. Two datasets D and D' are considered neighboring if they differ by exactly one row or one element. A randomized algorithm $\mathcal{M}$ guarantees ϵ -DP if, for all neighboring datasets D and D' , and for all subsets of possible outputs $S \subseteq \text{Range}(\mathcal{M})$, the following holds:

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D') \in S] \quad (2.2)$$

The parameter ϵ in the equation represents the privacy budget, which controls the strictness of the privacy guarantee. A smaller ϵ yields higher privacy but lower utility, whereas a larger ϵ relaxes the privacy bound. Importantly, DP is not an algorithm for achieving privacy, it is just a guarantee [12, 16, 17].

One of the foundational mechanisms to achieve ϵ -DP for numerical outputs is the Laplace Mechanism [12]. It operates by injecting random noise drawn from a Laplace distribution into the true, deterministic output of a function. The scale of this noise depends on the allocated privacy budget ϵ and the maximum possible change one record can cause, also known as the sensitivity [17].

2.2.2 Sensitivity

Sensitivity is important to determine the scale of noise needed to be injected by the Laplace Mechanism in order to hold the privacy guarantee of ϵ -DP [12]. For the Laplace Mechanism we need to determine the so called global L_1 -sensitivity, denoted as Δf . It represents the maximum possible magnitude by which a single element can change the outcome of a query function f [17]. For a function f and two neighboring datasets D and D' it is formally defined as:

$$\Delta f = \max_{D, D'} \|f(D) - f(D')\|_1 \quad (2.3)$$

The sensitivity provides a worst-case bound that is independent of the actual dataset being analyzed. In the context of the Laplace Mechanism, the random noise added to the query result is drawn from the distribution $\text{Lap}(\Delta f/\epsilon)$. Therefore, functions with high sensitivity require a larger scale of noise to mask the potential presence or absence of a single record, directly impacting the final data utility.

2.2.3 Composition and Post-Processing Theorems

When design complex or continuous privacy algorithms, two core properties of Differential Privacy are important [18, 24]:

- **Sequential Composition Theorem [18, 19]:** If an adversary observes k independent outputs from mechanisms $\mathcal{M}_1, \dots, \mathcal{M}_k$ on the same data, each satisfying ϵ_i -DP, the cumulative privacy leakage is bounded by $\sum_{i=1}^k \epsilon_i$. In streaming settings, unbounded composition poses a major risk, as continuous querying will eventually deplete the privacy budget ($\epsilon \rightarrow \infty$). In case of complete budget consumption, the resulting noise for not further weakening the privacy guarantee is unbounded (∞) and therefore destroying all data utility.
- **Post-Processing Theorem [18]:** If an algorithm $\mathcal{M}$ satisfies ϵ -DP, then any data transformation $g(\mathcal{M}(D))$ executed without direct access to the original dataset D remains ϵ -differentially private. This allows for structural optimizations, graph corrections, or filtering to occur after noise addition without weakening privacy guarantees.

2.2.4 Privacy Levels

The configuration of neighboring datasets defines the protection scope:

- **Event-Level Privacy [24, 18]:** Assumes neighboring logs D, D' differ by exactly one single event. Using event level privacy users can deny one single event but not more. Since users typically contribute an entire sequence of events (a trace) this privacy level is insufficient for our work.
- **User-Level Privacy [24]:** Assumes neighboring logs D, D' differ by an entire user's contribution (i.e., a full case/trace). A user can deny being part of the entire process or as many events as the user would like. This provides robust protection but significantly amplifies sensitivity, as one user can drastically alter event metrics.
- **w -Event Privacy [24]:** Tailored for streaming data, this relaxation protects all events within a moving window of size w . Instead of protecting an individual's presence across the entire infinite stream, w -event privacy restricts the protection scope to a sliding window of size w . It guarantees that any set of events occurring within a continuous window of w timestamps remains private. Two infinite event streams S and S' are considered w -neighboring if they differ only in events occurring within any single window of size w . It

binds the temporal memory of the mechanism, guaranteeing that an individual's presence inside any sliding window segment remains private. In other words, as user can deny sequences of events as long as they are inside a window of w consecutive timestamps.

3 Methodology

The goal of this work is to develop a methodology to continuously mine Directly-Follows Graphs from an event stream while strictly satisfying user-level Differential Privacy. In a non-private setting, a discovery algorithm would continuously update a DFG from the stream and make the graph available upon request [42]. However, ensuring DP in a continuous setting introduces significant challenges since infinite data streams can easily lead to unbounded sensitivity and fast privacy budget exhaustion [19]. To address these challenges, our methodology is structured as a continuous processing pipeline, as illustrated in Figure 3.1. We first establish a private graph representation. We then introduce a combination of frequency capping, temporal sliding windowing, Bloom filter gating, and trace truncation to strictly bound user sensitivity. With bounded sensitivity, we define mechanisms for managing the privacy budget over continuous publications with dynamic reclamation. Finally, we apply post-processing techniques to recover structural utility from the noised output before performing process model discovery.

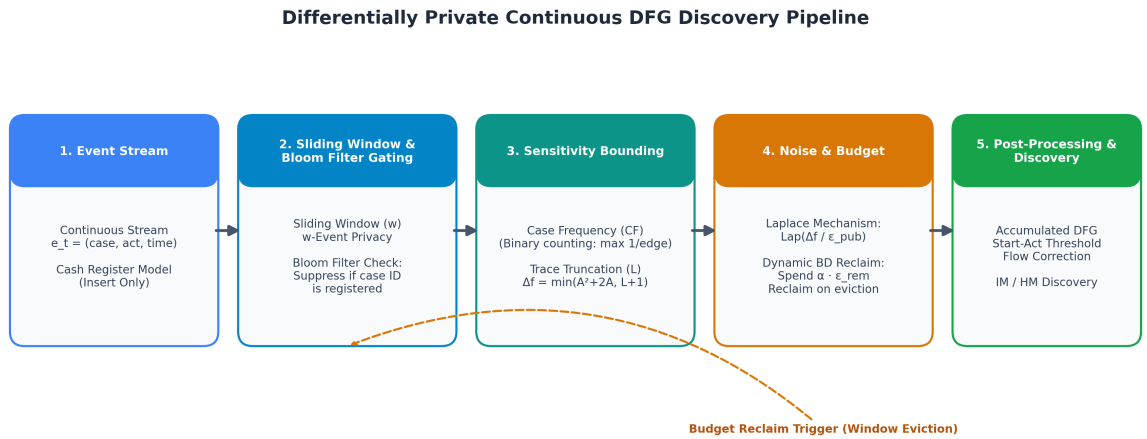


Figure 3.1: Architectural overview of the differentially private continuous DFG discovery pipeline, showing the data flow from event stream input to model discovery.

3.1 Graph Representation

The first step in our pipeline is to model the DFG in a manner compatible with standard DFG mechanisms. To achieve this, we represent the DFG as a histogram of counters, a data structure thoroughly researched and validated within standard DP practice [12]. Each unique directed edge in the graph thereby corresponds to a specific independent bucket in the histogram. Assuming a known universe of A unique activities, the maximum number of internal transitions between these activities is A^2 . Following the methodology of Schulze et al. [37], we introduce artificial dummy nodes a_{start} and a_{end} , adding A outgoing edges from a_{start} and likewise A incoming edges to a_{end} . Thus, our histogram monitors a counter total of:

$$|E| = A^2 + 2A \quad (3.1)$$

All counters are pre-initialized to zero. When satisfying DP via the Laplace mechanism, noise is injected into every counter in the histogram, including those representing transitions that never actually occurred. This necessary step (clamping) for privacy [12] potentially introduces false-positive edges into the resulting DFG.

3.2 Bounding User Influence: Sensitivity and Windowing

With the histogram model established, we must determine the scale of noise required, which is dictated by the global sensitivity Δf [17, 12]. In an unbounded event stream, a single user (case) could theoretically contribute an infinite number of events over an infinite amount of time. Establishing user-level privacy, where we want to protect all events of a case, this unbounded contribution results in infinite global sensitivity, requiring infinite noise and thereby destroying all data utility [19]. To counteract this, we systematically restrict a user's influence across three dimensions.

3.2.1 Per-Transition Bounding: Case Frequency

Conventional DFG computation relies on **Event Frequency (EF)** [2, 42], where every occurrence of a transition increments its corresponding counter. To bound a user's impact on any single counter, we draw inspiration from Schulze et al. [37] and adapt the computation to utilize **Case Frequency (CF)**, also known as binary counting. Under CF, a user can influence a specific edge counter at most once, regardless how often they repeat that transition. This strictly bounds the sensitivity of every single counter to exactly 1. However, a user could theoretically still traverse every distinct edge in the graph, leaving the worst-case global sensitivity across the histogram to:

$$\Delta f_{\text{CF}} = A^2 + 2A \quad (3.2)$$

For processes with a large activity space A , the global sensitivity therefore still remains very high.

3.2.2 Temporal Bounding: w -Event Privacy and Bloom Filter Gating

Beyond the number of transitions, we must also restrict a user's temporal presence in the stream. According to the Sequential Composition Theorem [18, 19], continuously updating and publishing graphs over time causes the cumulative privacy budget ϵ to sum towards infinity (for infinite publications). To resolve this, we adopt the sliding window concept of w -event privacy [24], ensuring that any window of w events is protected by at most ϵ -DP. To enforce this, we apply a strict architectural constraint: the influence of any event exiting the active window w

must be entirely removed from the DFG counters. When a new event arrives and the window is full, the oldest element must be evicted. Eviction removes the oldest element's influence from the DFG edge counters: the per-case edge count is decremented, and the global edge counter is reduced if the case no longer contributes to that edge. The start activity counter is adapted as well if the evicted event was the first event of its case. This ensures the DFG always reflects only the events currently in the window. Furthermore, to prevent cross-referencing across multiple windows, which would weaken privacy guarantees according to the Sequential Composition Theorem [19], all events belonging to the same user (case) must be confined to at most one window of w -events. This way, we expand the w -event privacy guarantee to user-level privacy, since all events of a specific user are confined to at most one window. We enforce this confinement using a Bloom Filter [8] (BF) gating mechanism. Before processing each event, the system checks whether the event's case ID attribute is present in the BF. If so, the event is suppressed (i.e. it is still pushed to the window in order to maintain correct window length accounting, but it does not influence any counters). A case ID is registered in the BF under two conditions. Firstly, when a case has contributed L events within the current window and therefore reached its trace limitation. Secondly, when the oldest event of a case is being evicted from the sliding window. This guarantees that all active events of a case are contained within a single window of size w , satisfying user-level privacy.

3.2.3 Global Bounding: Trace Truncation

To finally reduce the overall global sensitivity from $A^2 + 2A$ to a manageable level, we introduce a case length limitation. We restrict the length of any individual case within the window to exactly L events, where $L < w$. Because a case of length L contains at most $L - 1$ internal transitions, a user can influence at most $L - 1$ unique internal edges plus one start activity edge and one end activity edge (dummy edges). In total a user can therefore influence at most $L + 1$ edges in the whole DFG. This can fundamentally lower the global sensitivity of our continuous discovery mechanism to:

$$\Delta f = \min(A^2 + 2A, L + 1) \implies \Delta f \leq L + 1 \wedge \Delta f \leq A^2 + 2A \quad (3.3)$$

By selecting a suitable L based on background knowledge of the process (e.g. using the 75th quintile of observed trace lengths), we aggressively lower the sensitivity, thereby significantly reducing the magnitude of injected Laplace noise.

Figure 3.2 illustrates how traces interact with the sliding window w , trace limit L , and Bloom Filter gating across three representative processing scenarios:

- **Scenario 1 (Complete Trace):** The trace lies completely within the active sliding window w and within the trace limit L (represented by the upward curly brace). All events belonging to this case are processed (marked as dark red dots), updating Case Frequency edge counts and recording start and end dummy transitions.
- **Scenario 2 (Evicted / Overlapping Trace):** The trace is larger than the window ($w = L$). One event prior to the Window Start line was already evicted. Events within the active window w are processed (dark red dots). Upon eviction of the first event, the case ID is registered in the Bloom Filter, causing subsequent events past the Window End line to be suppressed (faint red dots).
- **Scenario 3 (Trace Truncation at L):** The trace fits inside the sliding window w , but its length exceeds the trace limit L . Trace events from the start of the case up to the end of the L curly brace are processed normally (dark red dots). Upon reaching event L , the case ID is registered in the Bloom Filter, suppressing all subsequent events past the L brace before the Window End line (faint red dots).



Figure 3.2: Event stream trace processing and gating scenarios: (1) Complete trace within window w and L brace; (2) Evicted trace extending past window boundary where subsequent events are suppressed after eviction; (3) Long trace truncated at limit L with subsequent events suppressed past the L brace.

Having bounded the sensitivity and isolated user influence to a single window, we must manage how the available privacy budget ϵ is distributed across all (potentially infinite) publications within that window [18]. Based on Kellaris et al. [24], we adopt a Budget Distribution (BD) strategy with a reclaim mechanism. At each publication, rather than spending a fixed portion of the initial budget, we spend a fixed fraction α of the remaining budget:

$$\epsilon_{\text{pub}} = \alpha \cdot \epsilon_{\text{remaining}}$$

Using the remaining budget (rather than the initial budget) as the base means that each successive publication uses a decreasing share, preventing complete budget exhaustion. The parameter α is typically set to $\alpha = 1/r$ where $r = w/P$ is the ratio of window size to publishing period, thereby allowing an equal budget distribution over all publications. To prevent indefinite operations, spent budget must be reclaimed once the data it protects exits the window. The key principle used in our research is that budget consumed at a publication point is associated with the event that has most recently been added to the window. When this event later exits the window (exactly w events later), the associated budget is reclaimed and added to the remaining budget. This strategy implements a dynamic budget lifecycle: budget is consumed upon publication, travels through the window with the data it protects, and gets reclaimed once that data is no longer in scope.

3.2.4 Publishing Schedule

The triggering of these publications follows one of two distinct schedules: in **Event-Count Based (ECB)**, a DFG snapshot is published after a fixed number of events have arrived, completely ignoring timestamps and other factors. Because the publishing period P is predefined

in terms of events, budget distribution is straightforward. Conversely, in **Time Based** publishing (TB), the effective publishing period P must be estimated based on prior knowledge of event arrival rates to ensure the budget is not depleted too much within a single w -event window.

3.2.5 Laplace Noise Injection

At each publication point, the Laplace mechanism is applied to the current DFG snapshot. For each edge counter c_e in the histogram, an independent noise sample is drawn:

$$\tilde{c}_e = c_e + \text{Lap}\left(\frac{\Delta f}{\epsilon_{\text{pub}}}\right) \quad (3.4)$$

where $\Delta f = \min(A^2 + 2A, L + 1)$ is the global L1-sensitivity and ϵ_{pub} is the privacy budget allocated for this particular publication. Any negative counts or noise artifacts resulting from Laplace noise injection are subsequently handled during post-processing (by the Post-Processing Theorem [19]).

3.3 Utility Improvement via Post-Processing

The restrictions chosen to be able to apply user-level DP, specifically trace truncation and temporal windowing, effectively reduce sensitivity but introduce utility loss independent of Laplace noise. When long traces are truncated or split across sliding windows, their initial segments fall out of the active window, resulting in a loss of true structural start activities. Similarly, identifying true end activities is difficult even in a non-private setting, as the continuous nature of the stream obscures whether a user's last observed activity truly is their final step. To recover some of the lost utility without violating DP, we apply three post-processing techniques. Because all of them operate on the already noised output without accessing the original data, they satisfy the Post-Processing Theorem and preserve the original privacy guarantees.

3.3.1 Accumulated DFG and Structural Correction

First, we maintain an Accumulated DFG. At each publication, the current noisy DFG (including dummy START/END edges) is added element-wise to a running sum. Because the injected noise is zero-mean [18, 17], transient noise variations tend to partially cancel out over long horizons, allowing major underlying process trends to emerge. To further refine the structure of this noisy Accumulated DFG and the continuously published snapshots, we apply two primary correction algorithms:

- **Start-Activity Filtering:** A technique introduced by the infrequent miner [25] for filtering infrequent behavior. It uses a threshold heuristic designed to mathematically reconstruct missing process entry points by detecting strong, structural trends emerging over time within the accumulated history.
- **Flow Correction:** We explore the idea that flow-conservation constraints can be used to post-process noisy directly-follows graphs. In particular, a flow-conservation (input equals output) optimization helps correct Laplace-perturbed edge weights and improve structural utility [22].

3.4 Summary Methodology

This chapter presented a comprehensive methodology for discovering Directly-Follows Graphs from continuous event streams under strict differential privacy guarantees. By decomposing

the problem into a sequential pipeline, we address the fundamental challenges of unbounded data streams and infinite sensitivity.

Specifically, we enforce a strict temporal boundary using a sliding window combined with Bloom Filter Gating, ensuring that any user appears in at most one window. Within this window, global sensitivity is decisively bounded through a combination of Case Frequency and Trace Truncation, lowering the scale of necessary Laplace noise. To ensure continuous operation without budget depletion, we employ a dynamic Budget Distribution strategy that reclaims spent privacy budget as events exit the window. Finally, we mitigate the inevitable utility loss caused by Laplace noise and structural truncation by applying three post-processing techniques: Accumulated DFG, flow correction, and start-activity filtering.

Together these mechanisms form an architecture for continuous privacy-preserving process discovery. The following chapter will detail its concrete implementation and empirically evaluate its performance, utility and structural stability across a diverse set of event logs.

4 Design and Implementation

This chapter describes the concrete system architecture, the simulation framework including private streaming algorithm and baselines, and the experimental design used to evaluate the methodology presented in the previous chapter.

4.1 System Overview

4.1.1 Streaming Pipeline

Our system implements a streaming pipeline built upon the observer pattern. A **Pausable Event Stream (PES)** replays all XES-format recorded event logs as a simulated live stream. The stream extends pm4py's [7] already implemented "LiveEventStream" structure with pause/ resume methods. Those have been implemented using treading locks, condition variables, and future tracking via a 'ThreadPoolExecutor'. This guarantees that during a 'pause()' or 'flush()' operation, the stream blocks until the event queue is empty and all actively executing observer threads have resolved, ensuring that all pending events are processed at every publication point. Multiple observers are registered on this stream and process event independently. The Private Streaming DFG Miner represents is one of the observer and it is it's own category and our core algorithm, representing a sliding window of events and producing DFG snapshots regularly. Next, there are the Non private baseline miners, both an accumulating and a windowed variant, maintaining identical DFG structures without noise injection, serving as peer comparison to assess the cost of differential privacy upon utility. Lastly, there is the log recorder category, very simple algorithms that just record which event already have been processed for log evaluation. Again, there is a windowed version, representing the window in the private algorithm and an accumulating version representing the events that have been seen since the start of stream observation. At configurable publication points, the stream is paused, all observers are queried for their current state and a comprehensive set of metrics is recorded. The stream is then resumed until the next publication point.

Private Miner

The private streaming DFG miner represents the core algorithm of this work and extends pm4py's [7] "StreamingAlgorithm" interface. It maintains several internal data structures. Firstly, it maintains a sliding window implemented as a double ended queue of fixed capacity w . Each element in the queue is a 5-tuple:

(case_id, activity, edge, is_suppressed, stored_budget)

If an event is rejected by the BF gate, it is still pushed into the window with the activity and edge value set to null. This ensures the window size w and time-shifting logic remain accurate without altering the underlying histograms. The 5th field in our five-tuple is used for budget reclamation, when a publication occurs, the spent budget is stored on the newest window element and reclaimed when that element exits the window. Secondly, a Bloom Filter [8] for the already known case detection gate is described in the methodology. Case IDs are added to this filter to uphold user-level privacy. First, if a case exceeds the configurable trace length limit L , its ID is added to prevent exceeding the calculated sensitivity. Second, during the eviction procedure, if an evicted element represents a start activity of a case, the case id is added to the filter. This guarantees that all events of a user are bounded within a single w -event time frame and ensures that the used memory is limited. The BF is size based on the total number of cases and configurable false-positive rate (default 1%, as the memory capacity needed grows towards infinity the lower the rate is). While a BF introduces a small probability of incorrectly suppressed events from cases that were never seen before, this false-positive rate is acceptably low and the privacy guarantees remain intact, as false-positive suppressed events only reduce utility but never weaken privacy. Thirdly, several maps are being maintained for state representation. A map of the number of non-suppressed events per case currently in the window called "case_event_count", one other mapping maintains the contribution to the binary counting mechanism of each case for each map and a "user_state" map that simply records the last activity and timestamp seen per case. Lastly, the "global_edge_count" histogram is maintained representing the core count-variable. The counters are incremented when a case first introduces an edge and decremented during eviction when a case no longer contributes to that edge. The eviction procedure reverses the influence of the oldest window element on all previously mentioned data structures and adds the id of the case to the BF gate.

Accumulated DFG

The accumulated DFG is maintained by pre-allocating the full $(A + 2)^2 \text{edgespace}$, including the dummy start and end activity nodes. At each publication point, the noisy DFG snapshot (again with dummy activities) is aggregated into the running sum (each edge is added to the corresponding sum of that edge). We provide two retrieval models, a 'normal' Accumulated DFG and a flow corrected "Clean Accumulated DFG". The normal model returns the direct accumulated sum and for utility improvement, we filter the start activities as inspired by the infrequent inductive miner [25]. The "Flow-corrected" model applies the flow optimization implemented in the cvxpy python module [13, 5] before returning the accumulated DFG, i.e. the edges are adapted that the weight of the incoming edges for each node is equal to the outgoing edges of the same node and the outgoing weight of the dummy start activity is optimized to be equal to the sum of ingoing weights of the dummy end node.

$$\sum e_{ingoing} = \sum e_{outgoing} \quad (4.1)$$

Furthermore, the optimization includes a strict non-negative condition ($e \geq 0$) to ensure valid DFG semantics. After the optimization succeeds, the dummy START and END activities are pruned, and floating point near zero edges are removed via a threshold. Those pre processing measures are according to the Post-Processing Theorem [18] and consequently do not weaken the privacy guarantee. In the remainder of this work, we will refer to the raw start activity filtered model as Accumulated DFG (ADFG) and to the flow corrected model as Clean Accumulated DFG (CADFG).

4.2 Experimental Framework

To evaluate the utility and performance of the private streaming pipeline, an experimental framework has been implemented. The system automatically derives hyperparameter grids, establishes ground-truth baselines and executes evaluations in parallel across multiple event logs.

4.2.1 Dynamic Parameter Grid Generation

Instead of relying on arbitrary, static parameters, the experimental runner dynamically drives parameter configurations based on the statistical properties of each input event log. The total number of events N and the trace length distribution inside of the log are analyzed prior to execution. The sliding window capacity w is defined as a fraction of the total log size (e.g. 1%, 5% and 10% of N) and can be adjusted. To evaluate user-level sensitivity, the trace limit parameter L is bound to the 75th, 90th and 100th quintiles of the log's trace length (of course 100th quintile means all events are inside of the event limit).

The framework supports both ECB and TB publication strategies. For ECB, the publication period P is determined by a ratio $r = w/P$, tested at values $r \in \{1, 5, 10\}$. If the ratio is equal to one, this means that in every event window there is one publication, else if the ratio is greater than one this means more than one publication per window, otherwise it means that a publication occurs in not every window. To ensure sustainable budget consumption as the window slides, the allocated budget fraction per publication is strictly tied to this ratio via $\alpha = 1/r$. Time-based publishing evaluates the stream at fixed calendar intervals (specifically 1-day, 5-day, and 10-day intervals) to simulate real-time data needs. The exact values for the publication frequency (both ECB as well as TB), window size and trace limit can be adapted in the source code.

To handle the resulting multidimensional parameter grid efficiently, the framework employs a multiprocessing architecture. The heavy, un-windowed baseline DFG is mined exactly once per log in the main process and shared with parallel worker processes via a copy-on-write memory fork. Every worker takes care of a own parameter combination.

4.2.2 Baseline Generation

To objectively measure the error introduced by both differential privacy and the architectural decision like sliding window mechanism and trace truncation, a ground truth baseline is established. During the initial preprocessing phase where the hyperparameters are established, the input XES log is chronologically sorted using a stable merge-sort algorithm. The framework then extracts the exact, offline DFG from entire sequence of events. This non-private offline DFG acts as the baseline against all noisy streaming snapshots are compared at every publication point.

Furthermore, a non private windowed baseline mirrors the private window DFG algorithm, i.e. same event processing (CF), event eviction etc., but no Laplace noise is injected. This way we can see the implications of the injected Laplace noise.

4.2.3 Evaluation Metrics & Process Discovery

At each publication point, the system records a large set of metrics to quantify utility changes. Graph-level structural recovery is measured using Mean Absolute Error (**MAE**) and Mean Relative Error (**MRE**) of the transition weights. The preservation of the underlying DFG structure (which true edges have been preserved and which not) is evaluated via Edge Recall, Edge Precision, and the resulting F1-score. Those means compare the presence of active edges against the DFG baseline.

Process Discovery and Preprocessing

While statistical metrics measure raw graph error, we try to assess practical utility by the ability to discover valid process models from the noisy data. The framework integrates pm4py's [7] Inductive Miner and Heuristic Miner to convert the published DFG's into Petri Nets, evaluating them using token-based replay metrics including Fitness, Precision, Generalization and Simplicity. This way of evaluation has been used other works too [40, 37].

However, while evaluating the snapshots using the IM, the discovery often failed. We figure, because of the injected noise the inductive miner is not able to find any cuts at all and not even apply a so called Fall-through. To solve this, a preprocessing step has been implemented prior the Inductive Miner. A dynamic, relative noise threshold is calculated based on the maximum edge weight in the current DFG snapshot ($\text{threshold} = \text{noise_thresh} \times \text{max_count}$). All edges falling below this threshold are pruned as artificial noise artifacts, and the surviving edge weights are rounded to nearest integer value. This step is inspired by the infrequent Inductive Miner [25].

In the next chapter, we will evaluate the results of the methodology using the previous described evaluation pipeline.

5 Evaluation and Discussion

In this chapter, we aim to empirically validate the effectiveness of our differentially private continuous DFG discovery pipeline using the evaluation framework described in the implementation chapter ¹. The evaluation is designed to systematically measure the fundamental trade-off between privacy guarantees and structural utility across various process mining scenarios. We first detail the hyperparameter combinations and event logs used for the experimental evaluation. Subsequently, we present the most significant findings in terms of ECB and TB mode and finally discuss whether we achieved the goal of designing a private, memory-efficient continuous discovery miner.

5.1 Experimental Setup

Our evaluation framework is designed to test the continuous private binary miner against a diverse set of event logs. By testing across a grid of real and synthetic event logs, we ensure our findings are generalizable.

The experiments vary the following parameters:

- **Window Size (w):** Set to 1%, 5%, and 10% of the total log length.
- **Publishing Frequency:** In the event-count-based (ECB) mode, the parameter r is varied between 1, 2, and 5 (publishing frequency per window). In the real-time mode, the publishing schedule is varied between 1 day, 5 days, and 10 days.
- **Maximum Trace Length (L):** We vary between the 75th, 90th, and 100th quintile (LQ75, LQ90, LQ100) of the trace lengths observed in the log.
- **Privacy Budget (ϵ):** Is always set exactly to 1.0.

These parameters are based on prior knowledge about the observed process. In order to set them accordingly for other processes, a certain degree of domain knowledge is mandatory. There is, of course, a much wider range the parameters can be set to. The publication frequency, for example, could be completely random to simulate the need for real-time data in some applications. We decided to use these specific parameter combinations to observe the effect of changing individual dimensions incrementally (e.g., changing parameter w to 10% instead of 1%).

¹The code and results of our evaluation framework can be found here:

<https://github.com/KuhlKrypto/Privacy-Preserving-Continuous-Process-Discovery>

Since we have $3 \cdot 3 \cdot 3 = 27$ different parameter settings and 2 different publication modes, we obtain a total of 54 results per event log. We evaluate the framework on the following event logs:

- BPIC 2012 O [14]
- Environmental Permit Log [10]
- Sepsis Cases Event Log [28]
- BPIC 2020 Event Log (Prepaid Travel Cost) [15]

For the continuous stream progression plots presented in Sections 5.2 and 5.3, we select the hyperparameter configuration for each event log that yielded the highest overall structural F1-score. This isolates peak miner performance over the course of the stream. The chosen parameters stays the same throughout every subsection of section 5.2 and in every subsection of 5.3 (the sections chosen parameters differ though). The reference quality for all accumulating models is the ground truth baseline, defined as the F1-score of a process model discovered offline from the entire event log with unrestricted, non-private access. This ground truth value is represented as a fixed horizontal line, representing the best achievable model quality for each miner type. To analyze the sensitivity across the complete hyperparameter grid ($w \times r$ and $w \times \text{schedule}$), we evaluate model quality heatmaps displaying the final publication F1-scores across all parameter combinations.

5.2 Evaluation Results - Event Count Based

In this section, we evaluate the performance of the DP Streaming Miner. The evaluation highlights several key trends regarding model types, operating modes and the impact of hyperparameter tuning. This section focuses on the ECB publication mode, while the subsequent section examines the TB publication mode.

Private Window Miner

Figure 5.1 displays the structural F1-score of the models derived entirely from isolated privacy windows, compared against an offline baseline processing the exact same limited temporal data without differential privacy. We display the hyperparameter combination that yielded the highest F1-score. Both the DP and baseline curves exhibit significant fluctuations throughout the stream, a direct consequence of the continuous forgetting mechanism inherent to the sliding window. The Heuristic Miner (HM) consistently achieves higher F1-scores than the Inductive Miner (IM) in both the private and the non-private setting. The DP HM curve follows a similar shape to the non-private baseline but consistently remains several percentage points below it, indicating that while the restricted data volume within the window is the primary bottleneck, the injected Laplace noise introduces an additional, non-negligible quality penalty. The DP IM model remains substantially below the Baseline IM curve throughout the entire stream, indicating that the Inductive Miner is considerably more sensitive to noise at the limited data volumes available within a single window. Notably, in some event logs such as the Environmental Permit Log, the IM fails to produce a valid model for several window publications entirely, leaving gaps in the progression curve.

Accumulating Models vs. Ground Truth

We distinguish three accumulating DFG variants used in this work. The Accumulating DFG is the raw, noisy DFG built by continuously accumulating differentially private event counts over

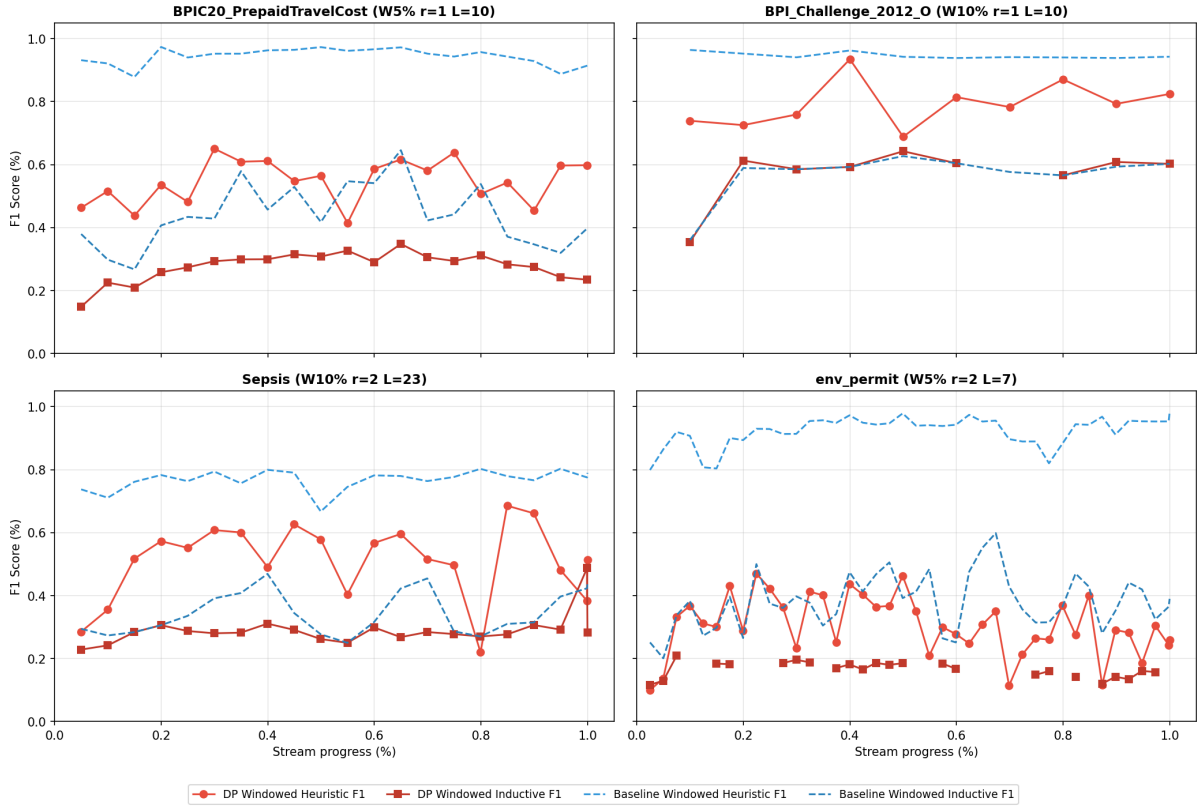


Figure 5.1: Comparison of the F1-scores for the Private Windowed DFG against the Offline Baseline Windowed DFG across all evaluated logs.

the stream. The Clean DFG applies the flow correction to the ADFG, restoring flow balance while preserving the accumulated counts. The SA filtered Accumulating DFG additionally applies start-activity filtering on top of the Clean DFG to remove spurious edges entering the process from unexpected start states.

Figure 5.3 illustrates the performance of the continuously accumulating DP process model relative to an original offline computed model (the real event log the stream processed), which has unrestricted, non-private access to the entire seen log up to that point. As the stream progresses, the F1-scores of the DP Accumulating models rise as the streaming of the event log proceeds. By the end of the data stream, the performance of the DP models converges almost perfectly with the baseline across all four logs in case of the inductive mined models. In case of the BPI Challenge 2012 O log, the graph representing the inductive miner surpasses the baseline. This surprising above baseline behaviour occurs because the Inductive miner (infrequent [25]) applies a noise threshold that filters out low frequency edges, which can produce structurally more simpler models which can in turn score higher F1-scores [25]. For the Heuristic Miner, the F1-score is rising throughout all observed event logs, but neither curve reaches the same level as the baseline. Both miners fail to mine a model in the beginning of the stream of most event logs (only exception is the Environmental Permit log).

Structural DFG F1 Scores

Figure 5.4 isolates the structural accuracy of the three intermediate Directly-Follows Graphs before they are passed to the discovery algorithms. All three DFG variants cluster at a similar structural F1-score level throughout the stream. The Accumulating DFG generally scores

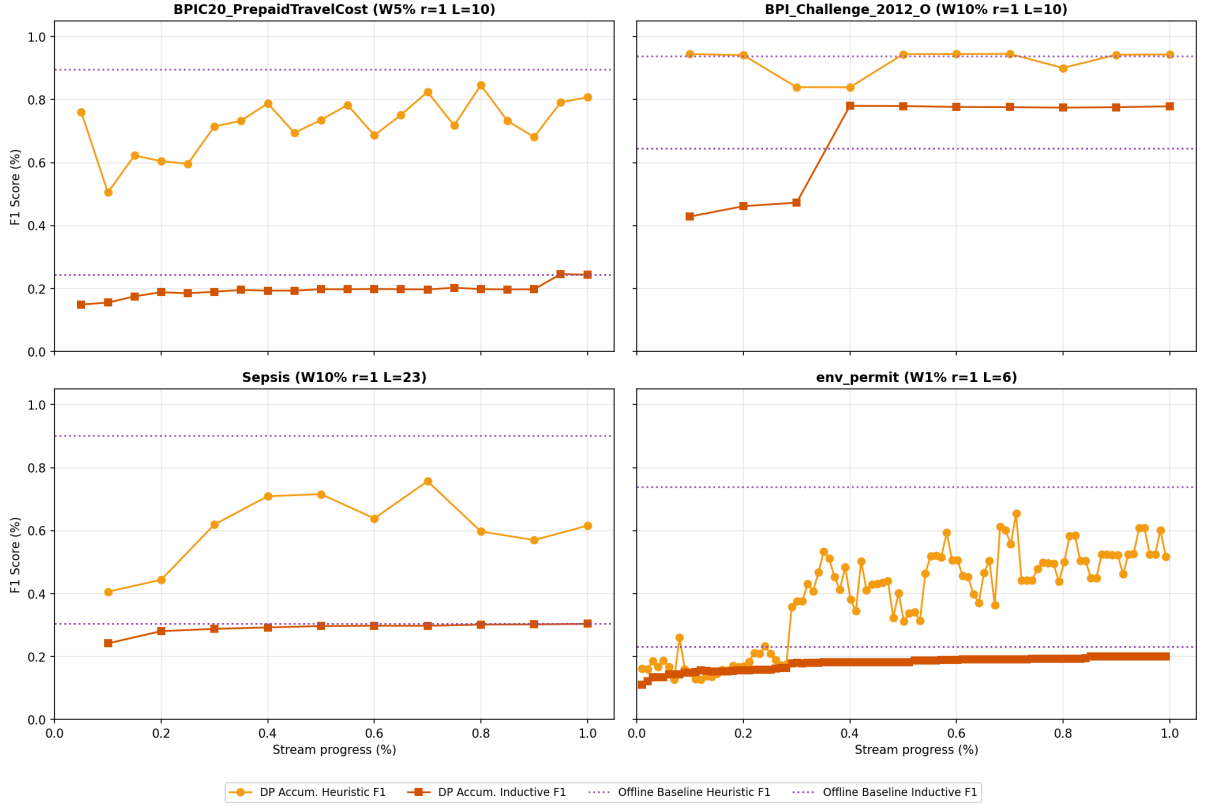


Figure 5.2: F1-score progression of the Accumulating DFG (ADFG) model compared to the static ground truth baseline (full-log, non-private).

marginally higher than the Windowed DFG, reflecting the benefit of accumulated event counts. Notably, the Filtered SA Accumulating DFG overlaps exactly with the Accumulating DFG across the entire stream, demonstrating that the start-activity filtering and flow-correction post-processing steps preserve the underlying structural integrity of the graph rather than distorting it. The Windowed DFG curve exhibits slightly higher volatility and on average a few percentage points lower accuracy due to the continuous forgetting mechanism.

Model Generality and Simplicity

In terms of generality, our evaluation results reveal a divergence between the two accumulating models. The Filtered SA Accumulating IM model achieves the highest generality. In case of the Sepsis Cases Event log it starts at ≈ 0.75 and converges monotonically to ≈ 0.93 by the end of the stream, almost exactly matching the ground truth IM generality (≈ 0.93). In contrast, the Accumulating IM model remains at substantially lower generality throughout, peaking at approximately 0.45 and staying well below the ground truth baseline. For the HM, both accumulating variants yield a similar, moderate generality trajectory, stabilizing at around 0.40–0.45 by stream end, still clearly below the baseline HM generality of ≈ 0.79 . The large gap between the IM variants underscores that the flow-correction step is critical for enabling the Inductive Miner to produce well-generalizing models. The simplicity metric behaves fundamentally differently from generality. Across all four model variants (Accum. IM, Accum. HM, Filtered SA Accum. IM, Filtered SA Accum. HM), simplicity remains exceptionally stable at approximately the initial publication to the end of the stream, matching the baseline simplicity level almost perfectly. This confirms that the differential privacy mechanisms in combination with our post processing techniques do not artificially inflate the structural complexity of the resulting process models and have no adverse effect on model simplicity. Visualizations of the generality

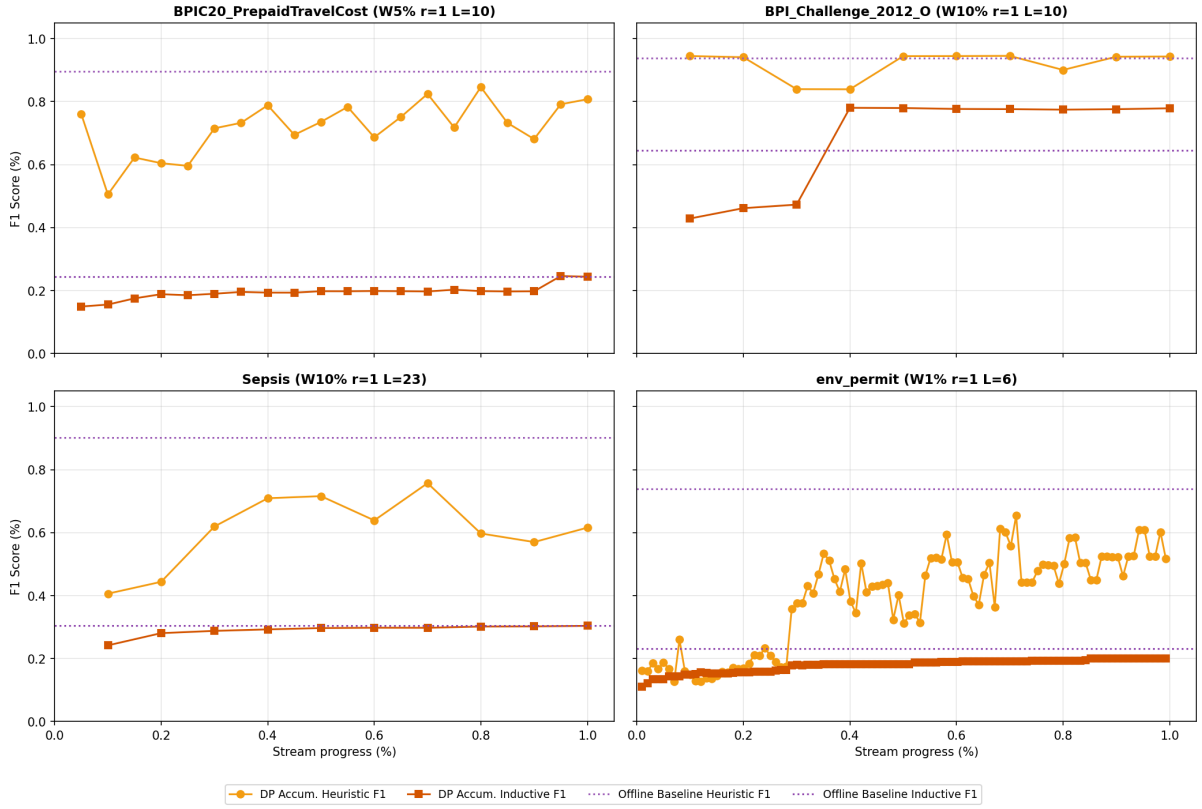


Figure 5.3: F1-score progression of the Clean DFG (CADFg) model compared to the static ground truth baseline.

and simplicity curves can be found in the Appendix.

Budget Decay and Noise Cost

Figure 5.5 displays the interaction between the privacy budget management and the corresponding noise cost. The cumulative ϵ spent monotonically increases as publications occur, constrained strictly by the global privacy budget allocation ($\epsilon = 1.0$). Simultaneously, the Mean Relative Error (MRE) tracks the proportional impact of the injected Laplace noise. For configurations with a high publication frequency (e.g., higher r values in BPIC20), the MRE tends to increase over time as the available budget fragments into smaller allocations per publication. In configurations where the entire window budget is consumed uniformly, the MRE remains stable or slightly decreases as the underlying true event frequencies accumulate. In ECB the epsilon budget consumption always stabilizes at a constant level over time.

5.3 Evaluation Results - Time Based Mode

When the streaming miner operates under a TB publication schedule, the dynamics differ noticeably from the ECB setting. Instead of publishing after a fixed number of events, the miner releases a model snapshot after a predefined time interval (1-day, 5-day or 10-day windows in our case). Consequently, the amount of data contained in each publication window can vary dramatically across logs, which in turn influences utility, noise and budget consumption.

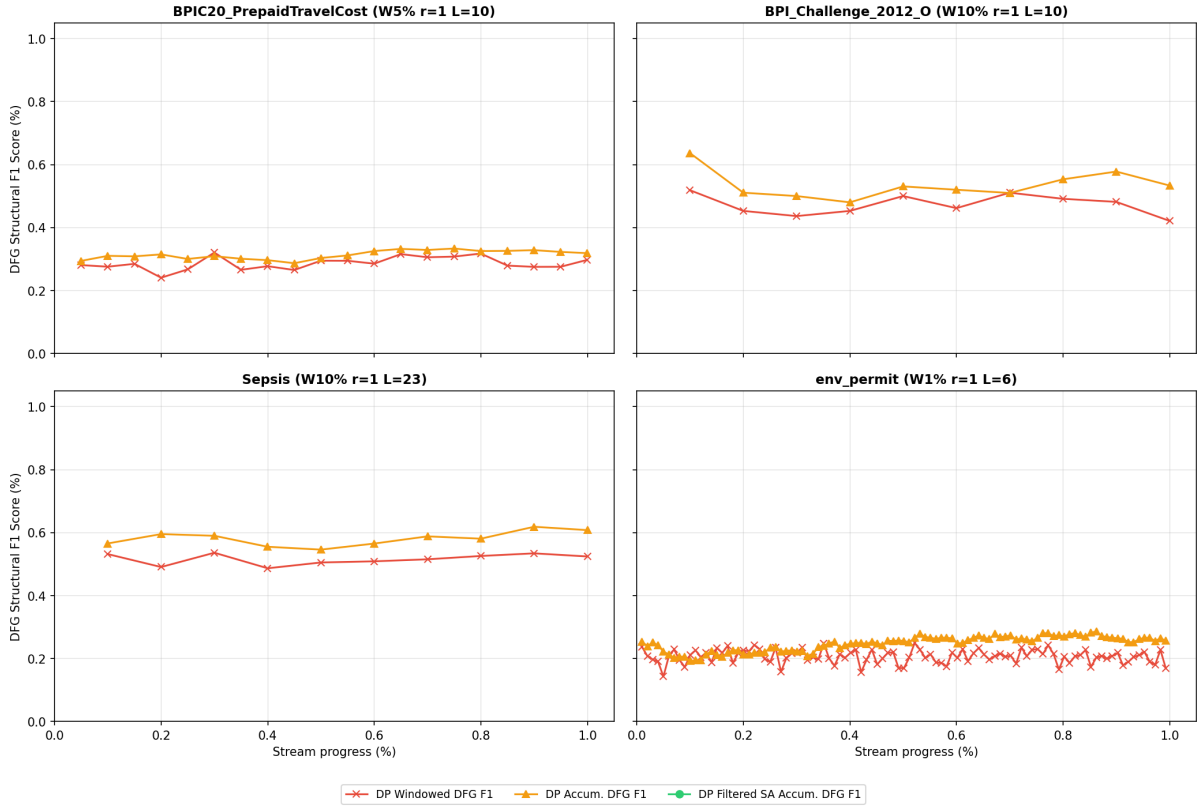


Figure 5.4: Structural F1 scores of the intermediate Directly-Follows Graphs (Windowed, Accumulating, and Filtered SA Accumulating).

5.3.1 Budget Consumption and Mean Relative Error (MRE)

In the ECB mode the cumulative ϵ consumption quickly reaches a plateau (see Fig. 5.5). In TB mode, the budget keeps fluctuating throughout the whole stream because every time-based publication consumes a fresh portion of the remaining budget, as explained in the implementation chapter. Unlike ECB, the budget can temporarily recover when the remaining budget replenishment exceeds consumption in calmer stream periods, but the dominant trend over the full log is a continued decrease until exhaustion. The MRE which quantifies the relative size of the added Laplace noise, remains low and stable during the main portion of the stream but explodes in every log in time of sparsely filled event window as the budget becomes exhausted. For the Sepsis event log under the 1-day interval configuration ($w = 5\%$, $L = 16$), the MRE of both the Windowed DFG and the Accumulating DFG remains below ≈ 500 (relative to the offline computed DFG) for most of the stream, but spikes to values of 30,000–50,000 in the final publications as the budget approaches zero and the allocated Laplace noise becomes enormous.

F1-Scores

Overall, the F1-scores in TB operation mode are lower than in ECB mode. The F1-Score curves of the Private Window DFG in general exhibit strong fluctuations. The heuristic miner yields in general higher F1-scores than the models discovered by the inductive miner; this observation matches the insights made in ECB mode. Regarding the Accumulating Models, the trend holds mostly. Sometimes the inductive miner yields better results than the heuristic one. This behavior occurs because the Inductive Miner focuses on identifying existing edges [25] and is therefore better suited for handling Laplace noise in terms of F1-score.

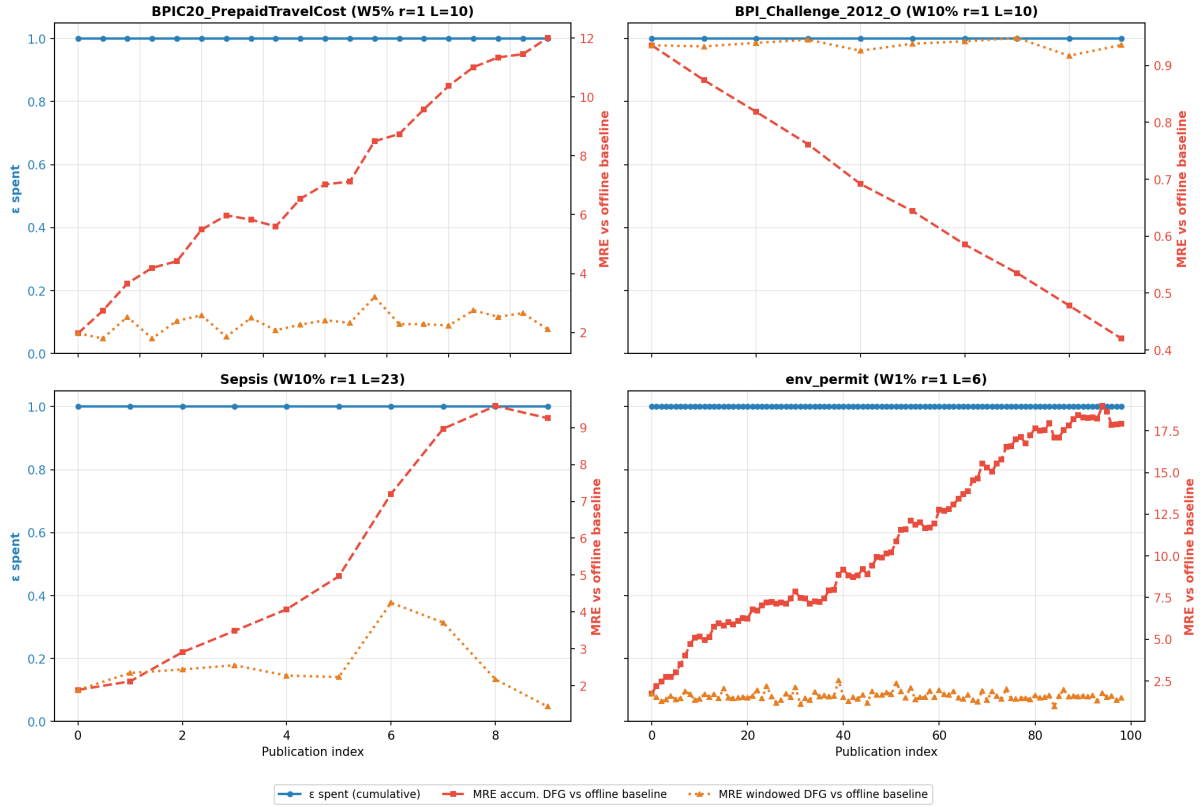


Figure 5.5: Budget decay in ECB mode ($r = 5$, $w = 10\%$, $L = 185$, Sepsis): cumulative ϵ spent and Laplace scale b (noise scale) over the publication index. Both stabilize after the initial window-filling phase.

Regarding the structural difference between the Accumulating and Clean Accumulating DFG, the Clean Accumulating DFG enables the Inductive Miner to produce sound models more consistently than the normal Accumulating one. Again, the underlying reason is that the Inductive Miner only produces sound models [25], which is easier to achieve with a flow-corrected DFG. Also, the scores assessed for the inductive mined model of the Clean Accumulating DFG often align with the reference model quality of the original log, while the HM version often misses the baseline by several percentage points.

Edge Metrics, Generality and Simplicity

The remaining metrics evaluated in ECB mode exhibit similar trends in TB mode. Edge-level precision and recall decrease in parallel with the overall F1-score. Generality metrics rise more slowly and do not reach the same level as for the corresponding parameter configuration in ECB. Regarding simplicity, the metric values remain virtually identical to ECB mode, staying close to the non-private baseline.

Summary TB Mode

The TB mode leads to non-stabilizing budget consumption, which in turn causes substantially higher MRE values than in ECB mode. Consequently, nearly all metrics drop in value level. The F1 scores of private and Accumulating DFGs drop as well as for edge metrics and generality. Only simplicity stays on roughly the same level. An interesting observation regards the CADFG vs. ADFG. While in ECB there is nearly no difference between those two, the CADFG is able

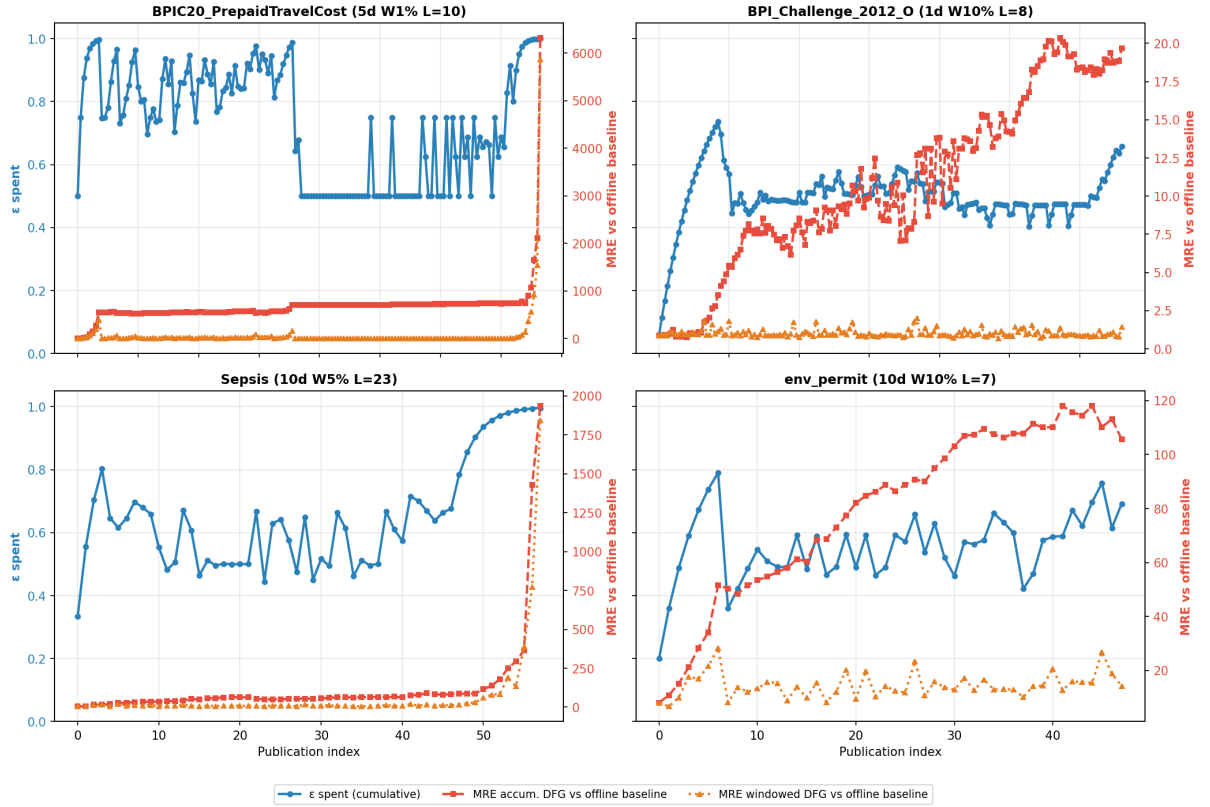


Figure 5.6: Budget decay and MRE in Time-Based mode. Unlike ECB mode, the cumulative ϵ spent does not stabilize but fluctuates throughout the stream, causing the MRE to explode near budget exhaustion or fluctuate too.

to produce valid models via IM more often than the ADFG. Also, the IM model of the Clean Accumulating DFG usually reaches the reference level of the baseline in the end of the stream. In the next section, we will discuss the observed behavior.

5.4 Parameter Sensitivity and Model Quality Heatmaps

To systematically analyze how hyperparameter choices impact final model utility across the entire parameter space, we evaluate faceted heatmaps mapping the final publication F1-scores across all combinations of window size $w \in \{1\%, 5\%, 10\%\}$, publication schedules ($r \in \{1, 2, 5\}$ for ECB and 1-day, 5-day, 10-day for TB), and maximum trace length quintiles $L \in \{LQ75, LQ90, LQ100\}$.

Figure 5.7 presents these faceted heatmaps for the BPI Challenge 2012 O event log, providing an overview of parameter combination results. The values reported in the heatmaps represent the final publication model F1-score (the harmonic mean of token-based replay fitness and precision evaluated against the accumulated log), discovered using the Heuristic Miner on the start-activity filtered and flow-corrected accumulated DFG.

The ECB heatmap (top panel) reveals the best overall performance at $w = 10\%$, $r = 1$, $L = LQ75$ with a final F1-score of 0.95, while trace limits combined with higher publication frequencies ($r = 5$) mostly yield lower F1-scores regardless of window size.

The TB heatmap (bottom panel) shows structural lower F1-scores across all configurations, with the highest observed value of 0.63 at $w = 10\%$, 5-day schedule, $L = LQ100$. The faceted heatmaps illustrate complex, multi-dimensional interactions between hyperparameter selection and discovered process model quality. Due to space constraints, the faceted heatmaps

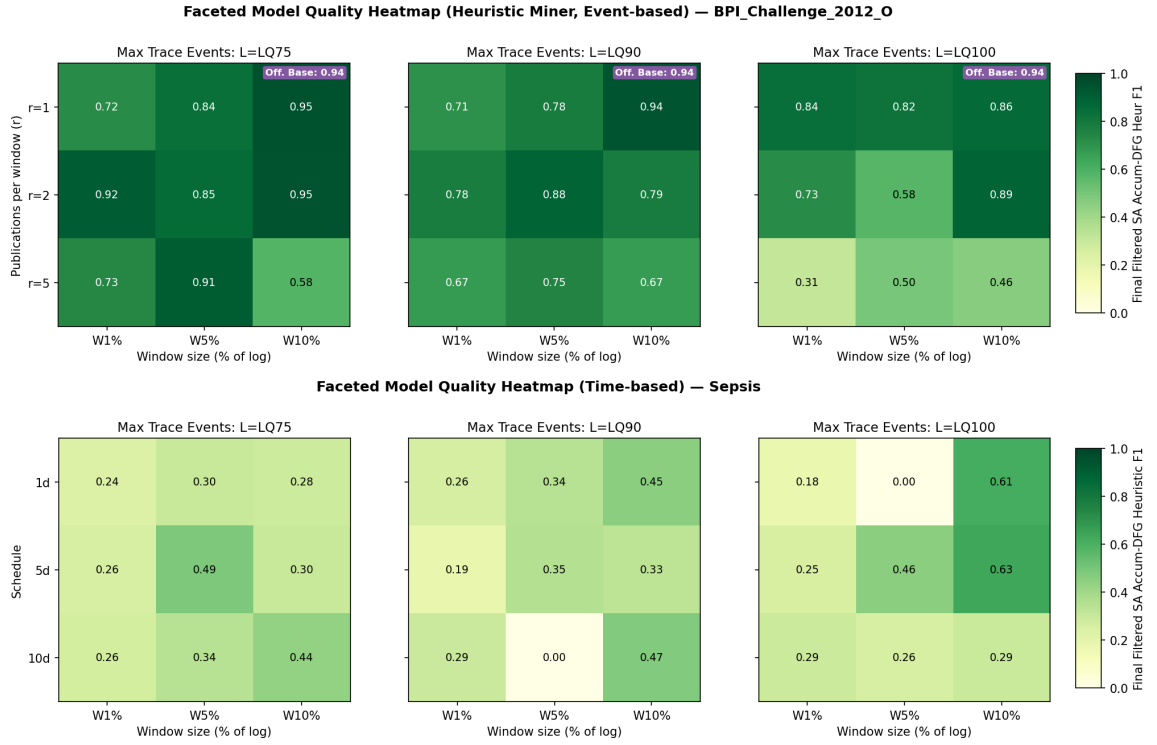


Figure 5.7: Faceted Model Quality Heatmaps for the BPI Challenge 2012 O across trace limit quintiles ($L \in \{LQ75, LQ90, LQ100\}$). Top: Event-Count Based (ECB) mode ($W \times r$). Bottom: Time-Based (TB) mode ($W \times \text{Schedule}$).

for the remaining event logs exhibit similar parameter interactions and can be found in the Appendix.

- **Publication Frequency / Time Schedule:** In ECB mode, lower publication frequencies consistently yield the highest F1-scores across all evaluated logs. Higher frequencies heavily fragment the privacy budget per window, confirming that budget splitting is a bottleneck for utility. In TB mode, very short intervals, generally perform poorly (with exceptions e.g. Sepsis Cases Event Log ($w = 10\%$, LQ100 and 1-day schedule), because the low event throughput per interval is easily overpowered by the injected noise. Longer schedules typically achieve better and more stable results.
- **Maximum trace length:** The benefit of capping trace length is visible through all observed event logs. Tighter trace limits usually outperform the full trace length. The surge in global sensitivity in some of the logs caused by preserving the longest outlier traces introduces Laplace noise, which degrades structural integrity of the discovered models far more than the structural loss caused by truncation.
- **Window size w :** The influence of the window size is not strictly monotonic and interacts heavily with the publication frequency and trace length. However, larger windows generally outperform the smallest window size in both ECB and TB modes. A larger window retains more events simultaneously, which helps to eclipse the fixed Laplace noise threshold, though this benefits diminishes if the publication frequency is set too high.

In the next section, we discuss the impact of these findings on utility and privacy, and evaluate whether we reached the goal of creating a continuous privacy-preserving DFG miner.

5.5 Discussion

In this section, we synthesize our experimental findings and reflect on the trade-offs between differential privacy guarantees, model utility, and memory efficiency in continuous process discovery.

5.5.1 Privacy vs. Utility Trade-off

Across all evaluation scenarios, the continuous miner strictly satisfies the differential privacy guarantee ($\epsilon = 1.0$). However, the utility achieved by the discovered process models varies significantly depending on the operating mode.

In ECB mode, the budget consumption stabilized quickly over time, which maintains a lower, stable Laplace noise level and allows the F1-scores of the accumulated models to converge towards the ground truth baseline. The Heuristic Miner achieves the closest convergence, reaching up to 0.86 F1 for the best ECB configuration on the Sepsis log, compared to the ground truth value of ≈ 0.90 . In TB mode, the budget does not stabilize automatically since each calendar-based snapshot consumes a different-sized portion of the privacy budget depending on event density. Short intervals (such as 1-day publishing) result in elevated and sometimes exploding noise near budget exhaustion, lowering model precision and generality. Conversely, longer intervals aggregate sufficient events per window to preserve more model utility while upholding the privacy guarantee, though even these configurations fall short of ECB-level quality.

Sensitivity and the Role of Trace Truncation

Bounding user sensitivity via trace length truncation introduces a structural trade-off that is always present: a lower L reduces the Laplace noise scale, but also truncates longer traces, potentially removing valid process pathways from the DFG. This trade-off cannot be cleanly isolated from the other hyperparameters in our evaluation, because w , r and L were varied simultaneously and only the final F1-score was recorded per configuration.

The faceted heatmaps (Fig. 5.7) illustrate this entanglement clearly for the BPI Challenge 2012 O event log. At $r = 1$, LQ75 yields the highest scores (0.72-0.95 depending on w), while LQ100 trails noticeably. This trend continues also for $r = 2$ and more extreme $r = 5$. Other observed event logs as for instance the Sepsis Case Event Log don't follow this exact pattern. The observed quality differences are therefore best understood as an interaction effect between L , w , and r , rather than a direct consequence of trace truncation alone.

Intuitively, the effect of L on noise is amplified when the per-publication budget fraction ϵ_{pub} is small (high r or short TB intervals), because noise already dominates the true counts. In these regimes, reducing sensitivity via a tighter L provides the largest relative noise reduction. For the Sepsis log specifically, the extreme skew in trace length distribution (average ≈ 14 events, maximum 185 [28]) means that LQ75 already covers the vast majority of traces while drastically reducing Δf , which explains why it tends to perform well. For logs with a more uniform trace length distribution, the same reasoning does not apply: a tight truncation bound would introduce structural bias without a proportional noise benefit.

Consequently, L cannot be tuned in isolation. Its optimal value depends on the interaction between trace length distribution, window size, and publication frequency, and should be treated as part of a joint hyperparameter selection rather than an independent design choice.

5.5.2 Post-Processing and Miner Robustness

An essential finding of our evaluation is the role of post-processing techniques in mitigating the impact of Laplace noise on intermediate DFGs. While raw noised DFGs often contain spurious negative or low-frequency noise edges, applying start-activity filtering and flow optimization restores flow consistency to the graph.

However, an unexpected finding in our evaluation was that even with CADFG post processing with and without extra start activity filtering, the Inductive Miner (infrequent) still occasionally fails to mine a valid process model (especially in TB mode). The inductive miner implementation in pm4py [7] follows the divide and conquer principle described by Leeman [26]. To our best knowledge, the concrete implementation fails to identify so called cuts and fall throughs because of unreachable edges, which can be created due to noise injection or the event eviction process. The last fall through 'FlowerModelDFG', generates an empty sub DFG that resolves to an unlabeled Process Tree which ultimately returns an unmineable process model. The combination of threshold filtering through the infrequent inductive miner and flow correction lead to more discovered models by the inductive miner, revealing that flow correction substantially improves graph correction and data sparsity remains as a bottleneck for discovery algorithms, especially the inductive miner.

In contrast, the Heuristic Miner almost always produces a valid model (but not necessarily a sound model [2]), but in case of the TB mode it exhibits higher sensitivity to injected Laplace noise and falls behind the F1-scores of the Inductive Miner (if it produces a valid model).

5.5.3 Memory Efficiency

By design, the streaming architecture maintains a strictly bounded memory footprint throughout execution. The system only stores the active sliding window of size w and the accumulated DFG represented as a fixed-size adjacency matrix of dimension $(A + 2)^2$. Consequently, memory consumption remains constant over time, regardless of the total stream length or stream velocity.

5.5.4 Limitations

Our experimental evaluation has several limitations that highlight scope constraints. Firstly, the hyperparameters must be set before the actual evaluation; this necessitates domain knowledge prior to mining models. If configured without proper domain knowledge, there is a substantial risk of miscalculation and degraded performance. Secondly, we used a static privacy budget of $\epsilon = 1.0$. While setting the privacy budget is policy-based and outside our control [12, 16], it is important to note that stricter privacy guarantees ($\epsilon < 1.0$) are often required in practice. If stricter privacy guarantees are demanded, the injected noise and consequently the error rate will rise even higher than reported in this work. This detrimental effect on utility intensifies proportionally as the budget decreases [12]. Thirdly, model quality assessment focused on structural metrics on the Petri net discovered out of the DFG (Fitness, Precision, Simplicity, Generality) have been measured using token-based replay. Alignment based fitness and precision were omitted due to performance bottlenecks in the current PM4PY [7] implementation.

5.5.5 Future Work

This research establishes a baseline for continuous private process discovery, opening several directions for future work. Firstly, the already mentioned limitations need to be investigated, e.g. evaluating model quality (fitness and precision) with Alignment based evaluation algorithms

or experimenting with stricter privacy constraints ($\epsilon < 1.0$). Secondly, we only focused on the most simple process model. In evaluation we had to rely on the conversion mechanisms in pm4py [7] in order to assess model utility. Future work needs to skip those conversion steps and try to discover more complex models as Process trees, Petri Nets or BPMNs [3] directly. Thirdly, there is always work to conduct which parameter combinations and relations have are good choices for event streams, especially if there is no much domain knowledge. Furthermore, this work relied on already known static event logs that we simulated to be event streams, it needs to be evaluated on real event streams in real world scenarios. Lastly, we experienced issues with the inductive miner discovering models from certain parameter configurations. There is a need for further post-processing techniques to further evaluate model utility.

5.5.6 Guidelines for Practitioners

For practitioners we try to present some guidelines on which model to choose and in which operating mode. The question whether to use ECB or TB is a question whether real-time data is needed. If it really is needed, you have to stick with TB and choose a publishing frequency based on your domain knowledge. Preferably try to match the frequency to the chosen window size, it should not be so small that only a few events or none get processed often, nor should it be so big that many events get lost and are not processed at all. If real-time data is not critical or the observed events occur on a reliable schedule, ECB is the way to go. Again, try to use a scheduling frequency which is as small as possible, but publishes at least once per event window. This way the used budget can be maximized without inferring too much noise. Regarding the trace length limit, we recommend evaluating the trace length distribution of the target log before mining. For logs with greater outliers (such as observed in BPI Challenge 2012 O), setting L to LQ75 yields a favorable noise reduction without disproportionate structural loss. For logs with a more uniform trace length distribution, LQ90 or LQ100 may be preferable to avoid unnecessary structural truncation. In terms of model quality, choose models discovered by the Inductive Miner if soundness is a main property. If it is more important to produce a valid (but not necessarily sound) model as often as possible, use the Heuristic Miner. ADFG and CADFG can both be used in ECB, when using the Inductive Miner in TB, the CADFG should be preferred, as it produces valid models more often and almost always reaches a comparable quality level. The private window miner can be used in both modes for a local view. Again, if a sound model is more important, choose the Inductive Miner, otherwise choose the Heuristic Miner.

5.5.7 Summary

Overall, the results demonstrate that designing a differentially private, memory-efficient continuous DFG discovery miner is feasible. Rather than relying on a single optimal parameter setting, utility depends on the balanced interplay between window size, publication schedule, and trace truncation limits. The choice of post-processing pipeline also plays a decisive role. The SA filtered ADFG substantially outperforms the raw ADFG in terms of generality, enabling the Inductive Miner to approach ground truth baseline model quality in ECB mode. When these parameters are chosen in accordance with stream characteristics, the streaming miner can produce process models whose structural quality approaches non-private offline baselines while maintaining strict differential privacy.

6 Related Work

Privacy challenges are not new to the field of process mining [1, 33]. Rafiei and van der Aalst introduced group-based privacy preserving tools in [34] for Process Mining, applying t -closeness and k -anonymity while focusing on the case perspective.

A similar technique was introduced by Fahrenkrog-Petersen et al. [21], who presented PRETSA, an algorithm that provides privacy guarantees through k -anonymity and t -closeness with a focus on the resource perspective while maintaining high utility for process model discovery. Additionally, Rafiei and van der Aalst [35] proposed methods for evaluating the effectiveness of privacy-preserving techniques in terms of sensitive data protection and utility preservation. Mannhardt et al. [29] laid foundational work by introducing Differential Privacy to static process mining, exploring trace truncation and frequency noise to sanitize static event logs. More similar to this study is the work done by Schulze et al. [37], who established a differentially private inductive miner (DPIM) for static event logs using case-frequency counting.

Rafiei et al. analyzed in [33] the privacy risk of continuously publishing anonymized data, showing that correspondence attacks on multiple releases can significantly weaken guarantees granted by group-based privacy preserving techniques. Their work highlights the privacy challenges that arises through repeated data releases, which are also central to differential privacy in streaming settings. Regarding streaming settings, Dwork et al. [19] formulated the theoretical foundation for applying DP in continuous data streams, showing how noise scales over long horizons using the Binary Mechanism. While Dwork et al. [17] addressed foundational continuous differential privacy, Kellaris et al. [24] introduced the concept of w -event privacy over infinite data streams. Their methodology bridges the gap between event-level and user-level privacy by protecting any sequence of events occurring within a sliding window of w consecutive time steps. This temporal bounding of the privacy mechanism is highly relevant for adapting streaming process discovery algorithms to privacy-preserving architectures, as it prevents continuous querying from completely depleting the privacy budget.

Erlingson et al. introduced RAPPOR [20] in 2014 as a differential private crowdsourcing application on streaming data. In RAPPOR they used a combination of Bloom Filters [8] and Randomized Response [18] to ensure local differential privacy [18]. Our proposed mining algorithm uses bloom filters as well, but only ensures central differential privacy, i.e. a data curator gather all data before applying a differential private algorithm.

7 Conclusion

This work investigated the feasibility of privacy preserving continuous process discovery from streaming event data.

To resolve these challenges, we developed the Private Window DFG Miner, a streaming pipeline architecture that guarantees user-level Differential Privacy while maintaining bounded memory footprint and model utility. Our pipeline addresses continuous privacy preserving process discovery through a coordinated sequence of architectural mechanisms. We restricted user influence on three dimensions in order to limit global sensitivity and therefore noise scale. Firstly, we replaced Event Frequency with Case Frequency counting, limiting a user's impact on a single DFGs edge counter to exactly 1. Secondly, we applied w event temporal sliding windows combined with Bloom Filter gating, which confines all events with the same case identifier to one and the same active window. Thirdly, we introduce trace truncation at length L , binding the maximum global L1-sensitivity to $\Delta f = \min(A^2 + 2 * A, L + 1)$.

To prevent budget exhaustion over infinite publications in one active w event window, we adopt a dynamic budget distribution strategy. Privacy budget allocated at one publication point travels through the sliding window alongside the data it protects and is reclaimed once all protected data exits the event window.

Operating under the DP Post-Processing Theorem, we maintain a Accumulating DFG that cancels out zero-mean Laplace noise over time. We complement this with start activity filtering and flow-conservation optimization to restore graph balance.

We evaluated our framework across 54 hyperparameter configurations per event log across four diverse real-world logs: BPIC 2012 O, Environmental Permit, Sepsis Cases, and BPIC 2020 Prepaid Travel Cost. Our empirical results yielded several key insights. Firstly, Event-Count-Based (ECB) publishing provides stable budget consumption over time. As event data accumulates, the structural F1-score of DP accumulating models steadily rises and converges almost perfectly with the non-private ground truth baseline for models discovered by the Inductive Miner. Models discovered by the Heuristic Miner yield similar results, but do not align as well with the baseline. In contrast, TB publishing leads to non-stabilizing budget decay, as sparse calendar windows expend privacy budget over few events, elevating the Mean Relative Error. Secondly, we demonstrated that structural model quality is governed by a highly entangled interaction between window size (w), publication frequency (r), and trace truncation (L). Setting these hyperparameters requires solid domain knowledge: while bounding outlier traces with a tighter L universally reduces noise, higher publication frequencies fragment the budget and deteriorate utility.

Overall, we have shown that continuous, privacy-preserving process discovery is both computationally feasible and memory-efficient. However, the drop in utility under sparse, time-based publication schedules remains a challenge.

Bibliography

- [1] Wil van der Aalst et al. "Process Mining Manifesto". In: *Business Process Management Workshops*. Ed. by Florian Daniel, Kamel Barkaoui, and Schahram Dustdar. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 169–194. ISBN: 978-3-642-28108-2.
- [2] Wil M. P. van der Aalst. "Foundations of Process Discovery". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Cham: Springer International Publishing, 2022, pp. 37–75. ISBN: 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3_2. URL: https://doi.org/10.1007/978-3-031-08848-3_2.
- [3] Wil M. P. van der Aalst. "Process Mining: A 360 Degree Overview". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Cham: Springer International Publishing, 2022, pp. 3–34. ISBN: 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3_1. URL: https://doi.org/10.1007/978-3-031-08848-3_1.
- [4] Arya Adriansyah et al. "Measuring precision of modeled behavior". In: *Information Systems and e-Business Management* 13 (Jan. 2014). DOI: 10.1007/s10257-014-0234-7.
- [5] Akshay Agrawal et al. "A rewriting system for convex optimization problems". In: *Journal of Control and Decision* 5.1 (2018), pp. 42–60.
- [6] Alessandro Berti and Wil M. P. van der Aalst. "A Novel Token-Based Replay Technique to Speed Up Conformance Checking and Process Enhancement". In: *Transactions on Petri Nets and Other Models of Concurrency XV*. Ed. by Maciej Koutny, Fabrice Kordon, and Lucia Pomello. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021, pp. 1–26. ISBN: 978-3-662-63079-2. DOI: 10.1007/978-3-662-63079-2_1. URL: https://doi.org/10.1007/978-3-662-63079-2_1.
- [7] Alessandro Berti, Sebastiaan van Zelst, and Daniel Schuster. "PM4Py: A process mining library for Python". In: *Software Impacts* 17 (2023), p. 100556. ISSN: 2665-9638. DOI: <https://doi.org/10.1016/j.simpa.2023.100556>. URL: <https://www.sciencedirect.com/science/article/pii/S2665963823000933>.
- [8] Burton H. Bloom. "Space/time trade-offs in hash coding with allowable errors". In: *Commun. ACM* 13.7 (July 1970), pp. 422–426. ISSN: 0001-0782. DOI: 10.1145/362686.362692. URL: <https://doi.org/10.1145/362686.362692>.
- [9] J. C. A. M. Buijs, B. F. van Dongen, and W. M. P. van der Aalst. "Quality Dimensions in Process Discovery: The Importance of Fitness, Precision, Generalization and Simplicity". In: *International Journal of Cooperative Information Systems* 23.01 (2014), p. 1440001. DOI: 10.1142/S0218843014400012. eprint: <https://doi.org/10.1142/S0218843014400012>. URL: <https://doi.org/10.1142/S0218843014400012>.

- [10] JCAM Buijs. "Receipt phase of an environmental permit application process ('WABO'), CoSeLoG project". In: *Eindhoven University of Technology* (2014).
- [11] Andrea Burattin. "Streaming Process Mining". In: *Process Mining Handbook*. Ed. by Wil M. P. van der Aalst and Josep Carmona. Cham: Springer International Publishing, 2022, pp. 349–372. ISBN: 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3_11. URL: https://doi.org/10.1007/978-3-031-08848-3_11.
- [12] Damien Desfontaines. *Differential privacy in practice (easy version)*. <https://desfontaines.blog/differential-privacy-in-practice.html>. Ted is writing things (personal blog). Nov. 2018.
- [13] Steven Diamond and Stephen Boyd. "CVXPY: A Python-embedded modeling language for convex optimization". In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5.
- [14] Boudewijn van Dongen. *BPI Challenge 2012*. 2012. DOI: 10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f.
- [15] Boudewijn van Dongen. *BPI challenge 2020 (2020)*. https://data.4tu.nl/collections/BPI_Challenge.2020/5065541/1.
- [16] Cynthia Dwork. "Differential Privacy: A Survey of Results". In: *Theory and Applications of Models of Computation*. Ed. by Manindra Agrawal et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 1–19. ISBN: 978-3-540-79228-4.
- [17] Cynthia Dwork and Aaron Roth. "The Algorithmic Foundations of Differential Privacy". In: *Found. Trends Theor. Comput. Sci.* 9.3–4 (Aug. 2014), pp. 211–407. ISSN: 1551-305X. DOI: 10.1561/04000000042. URL: <https://doi.org/10.1561/04000000042>.
- [18] Cynthia Dwork et al. "Calibrating Noise to Sensitivity in Private Data Analysis". In: *Theory of Cryptography*. Ed. by Shai Halevi and Tal Rabin. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 265–284. ISBN: 978-3-540-32732-5.
- [19] Cynthia Dwork et al. "Differential privacy under continual observation". In: *Proceedings of the Forty-Second ACM Symposium on Theory of Computing*. STOC '10. Cambridge, Massachusetts, USA: Association for Computing Machinery, 2010, pp. 715–724. ISBN: 9781450300506. DOI: 10.1145/1806689.1806787. URL: <https://doi.org/10.1145/1806689.1806787>.
- [20] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response". In: *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. CCS '14. Scottsdale, Arizona, USA: Association for Computing Machinery, 2014, pp. 1054–1067. ISBN: 9781450329576. DOI: 10.1145/2660267.2660348. URL: <https://doi.org/10.1145/2660267.2660348>.
- [21] Stephan A. Fahrenkrog-Petersen, Han van der Aa, and Matthias Weidlich. "PRETSA: differential privacy in process mining". In: *Process Mining Workshops (ICPM 2019)*. Springer, 2020, pp. 283–295.
- [22] Michael Hay et al. "Boosting the accuracy of differential privacy queries using linear constraints". In: *Proceedings of the VLDB Endowment* 3.1-2 (2010), pp. 1021–1032.
- [23] "IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams". In: *IEEE Std 1849-2023 (Revision of IEEE Std 1849-2016)* (2023), pp. 1–55. DOI: 10.1109/IEEESTD.2023.10267858.
- [24] Georgios Kellaris et al. "Differentially private event sequences over infinite streams". In: *Proc. VLDB Endow.* 7.12 (Aug. 2014), pp. 1155–1166. ISSN: 2150-8097. DOI: 10.14778/2732977.2732989. URL: <https://doi.org/10.14778/2732977.2732989>.
- [25] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. "Discovering Block-Structured Process Models from Event Logs Containing Infrequent Behaviour". In: *Business Process Management Workshops*. Ed. by Niels Lohmann, Minseok Song, and Petia Wohed. Cham: Springer International Publishing, 2014, pp. 66–78. ISBN: 978-3-319-06257-0.

- [26] Sander JJ Leemans et al. "Robust Process Mining with Guarantees." In: *BPM (Dissertation/Demos/Industry)*. Springer. 2018, pp. 46–50.
- [27] Volodymyr Leno et al. "Discovering process maps from event streams". In: *Proceedings of the 2018 international conference on software and system process*. 2018, pp. 86–95.
- [28] Felix Mannhardt. "Sepsis Cases - Event Log". In: (2016). DOI: 10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460.
- [29] Felix Mannhardt et al. "Privacy-Preserving Process Mining". In: *Business & Information Systems Engineering* 61.5 (Oct. 1, 2019), pp. 595–614. ISSN: 1867-0202. DOI: 10.1007/s12599-019-00613-3. URL: <https://doi.org/10.1007/s12599-019-00613-3> (visited on 11/05/2024).
- [30] Felix Mannhardt et al. "Privacy-Preserving Process Mining: F. Mannhardt et al." In: *Business & Information Systems Engineering* 61.5 (2019), pp. 595–614.
- [31] Jorge Muñoz-Gama and Josep Carmona. "A Fresh Look at Precision in Process Conformance". In: *Business Process Management*. Ed. by Richard Hull, Jan Mendling, and Stefan Tai. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 211–226. ISBN: 978-3-642-15618-2.
- [32] S. Muthukrishnan. "Data streams: algorithms and applications". In: *Found. Trends Theor. Comput. Sci.* 1.2 (Aug. 2005), pp. 117–236. ISSN: 1551-305X. DOI: 10.1561/0400000002. URL: <https://doi.org/10.1561/0400000002>.
- [33] Majid Rafiei and Wil M. P. van der Aalst. "Privacy-Preserving Continuous Event Data Publishing". In: *Business Process Management Forum*. Ed. by Artem Polyvyanyy et al. Cham: Springer International Publishing, 2021, pp. 178–194. ISBN: 978-3-030-85440-9.
- [34] Majid Rafiei and Wil M. P. van der Aalst. "TLKC-privacy model for process mining". In: *Dynamic Logic. Change, Action, and Intention (DALI 2019)*. Springer, 2019, pp. 198–216.
- [35] Majid Rafiei, Henrik Leopold, and Wil M. P. van der Aalst. "Towards quantifying privacy in process mining". In: *Process Mining Workshops (ICPM 2020)* (2021), pp. 185–197.
- [36] Hendrik Reiter et al. "Distributed Event Factory: A Tool for Generating Event Streams on Distributed Data Sources". In: *ICPM Doctoral Consortium and Demo Track 2024*. Workshop Proceedings. ARRAY(0x557ba2e91f40), Oct. 2024. URL: <https://ceur-ws.org/Vol-3783/paper%5C%5f323.pdf>.
- [37] Max Schulze et al. "Differentially Private Inductive Miner". In: *2024 6th International Conference on Process Mining (ICPM)*. IEEE, Oct. 2024, pp. 89–96. DOI: 10.1109/icpm63005.2024.10680684. URL: <http://dx.doi.org/10.1109/ICPM63005.2024.10680684>.
- [38] Wil Van Der Aalst and Wil van der Aalst. *Data science in action*. Springer, 2016.
- [39] Borja Vázquez-Barreiros, Manuel Mucientes, and Manuel Lama. "ProDiGen: Mining complete, precise and minimal structure process models with a genetic algorithm". In: *Information Sciences* 294 (2015). Innovative Applications of Artificial Neural Networks in Engineering, pp. 315–333. ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2014.09.057>. URL: <https://www.sciencedirect.com/science/article/pii/S0020025514009694>.
- [40] Maximilian Weisenseel et al. "Filtering at the Edge: Exploring the Privacy-Utility Trade-Off". In: ().
- [41] Alan F Westin. "Privacy and freedom". In: *Washington and Lee Law Review* 25.1 (1968), p. 166.
- [42] S.J. van Zelst. "Process mining with streaming data". English. Proefschrift. Phd Thesis 1 (Research TU/e / Graduation TU/e). Mathematics and Computer Science, Mar. 2019. ISBN: 978-90-386-4699-2.

8 Appendix



Figure 8.1: Faceted Heatmaps detailing the result of all assessed parameter configurations on models on the BPIC20 PrepaidTravelCost Event Log mined by the Heuristic Miner



Figure 8.2: Faceted Heatmaps detailing the result of all assessed parameter configurations on models on the Environmental Permit Event Log mined by the Heuristic Miner

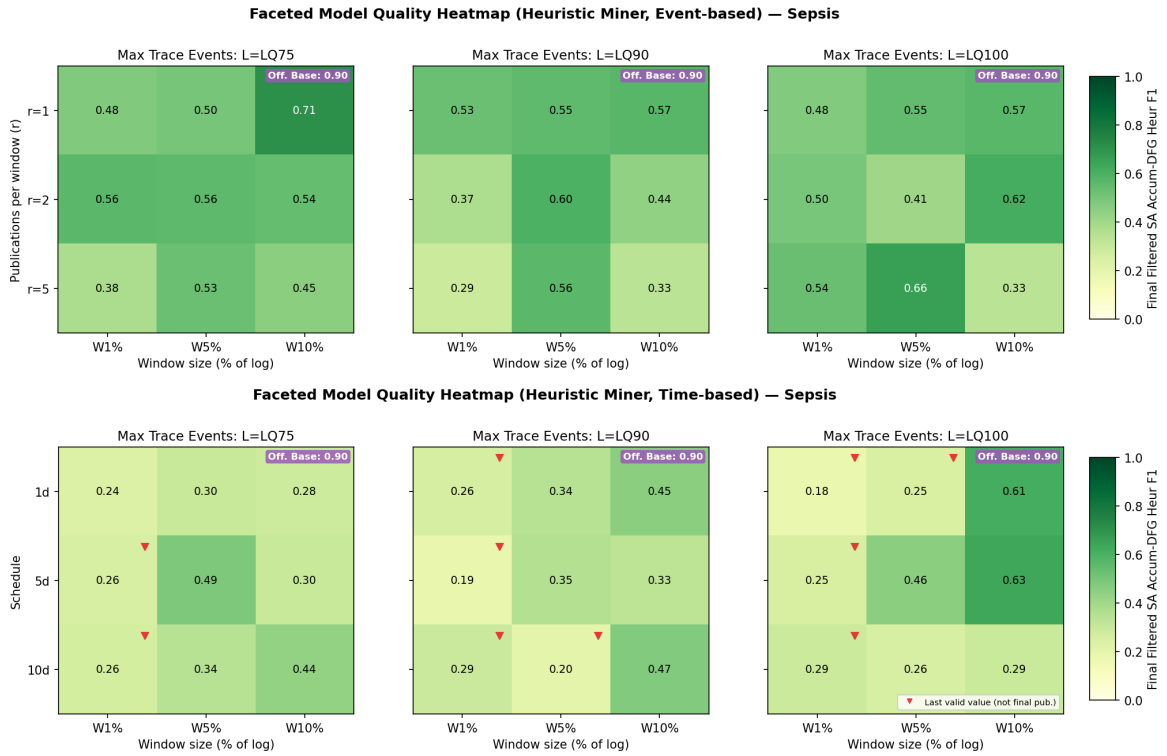


Figure 8.3: Faceted Heatmaps detailing the result of all assessed parameter configurations on models of the Sepsis Cases Event Log mined by the heuristic miner

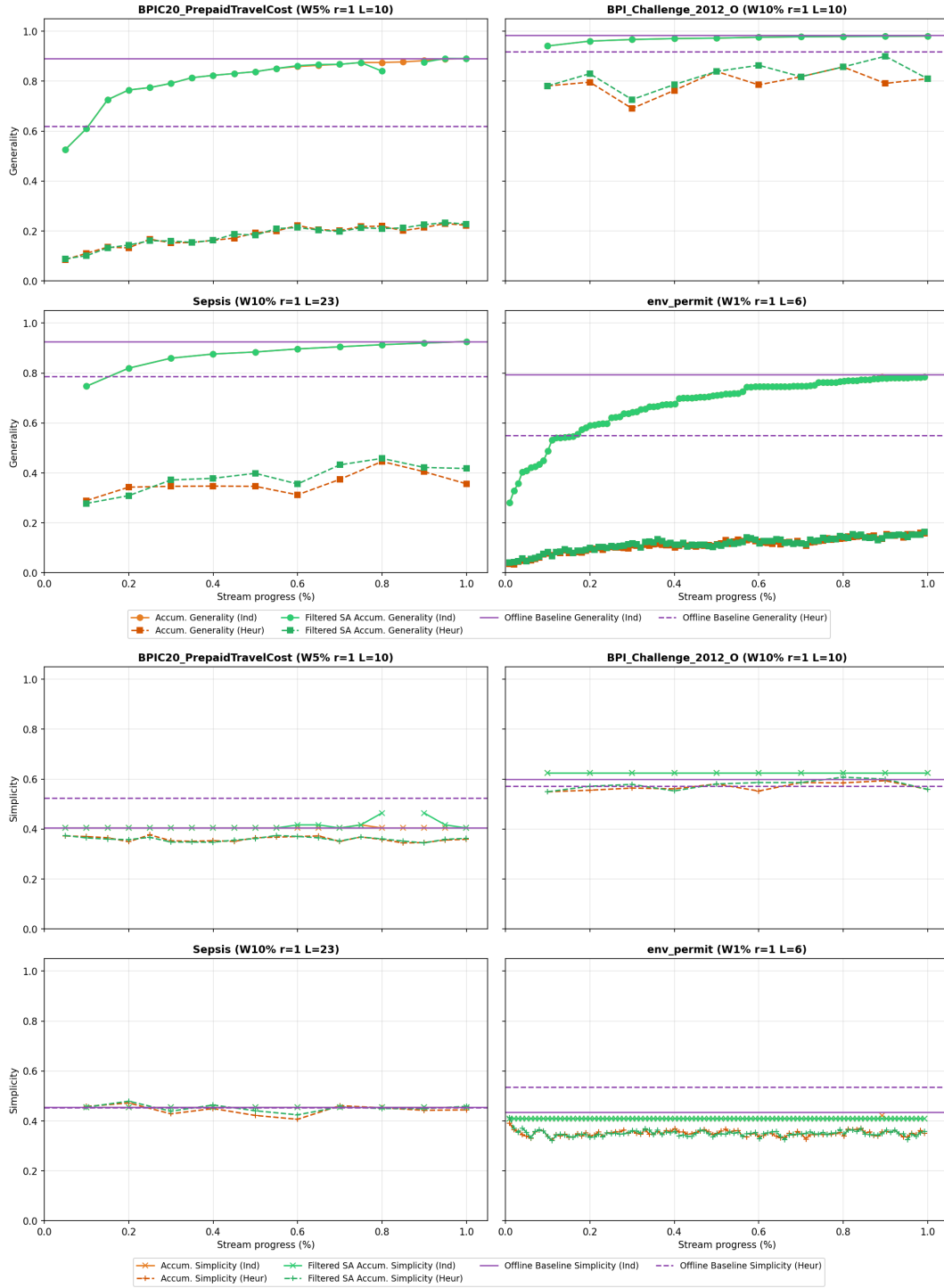


Figure 8.4: Generality and Simplicity Results of the observed event logs on models discovered by the Heuristic Miner